



National Technical
University of Athens

School Of Electrical and
Computer Engineering

Internet Applications

Server-side Programming and REST services

Dr. Chondrogiannis Efthymios

Introduction

- Web Application Development

- Client-side Programming

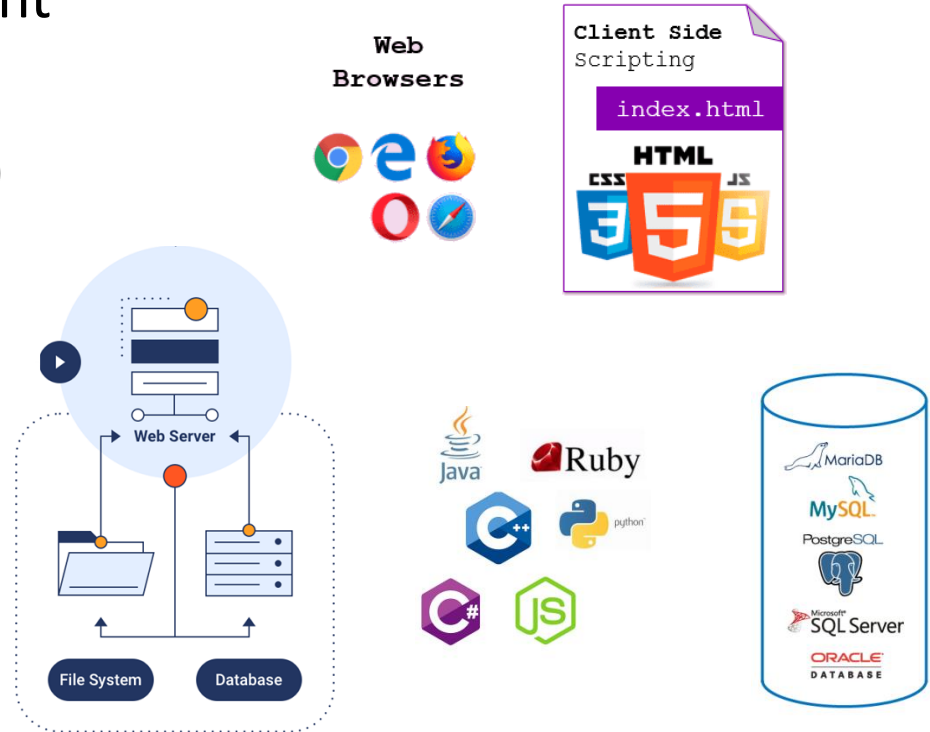
- Web Browser (and JS engine)
 - HTML, CSS and JavaScript

- Server-side Programming

- Web Server
 - Java/Python Programs, DBs

- HTTP protocol

- Data Exchange: XML & JSON



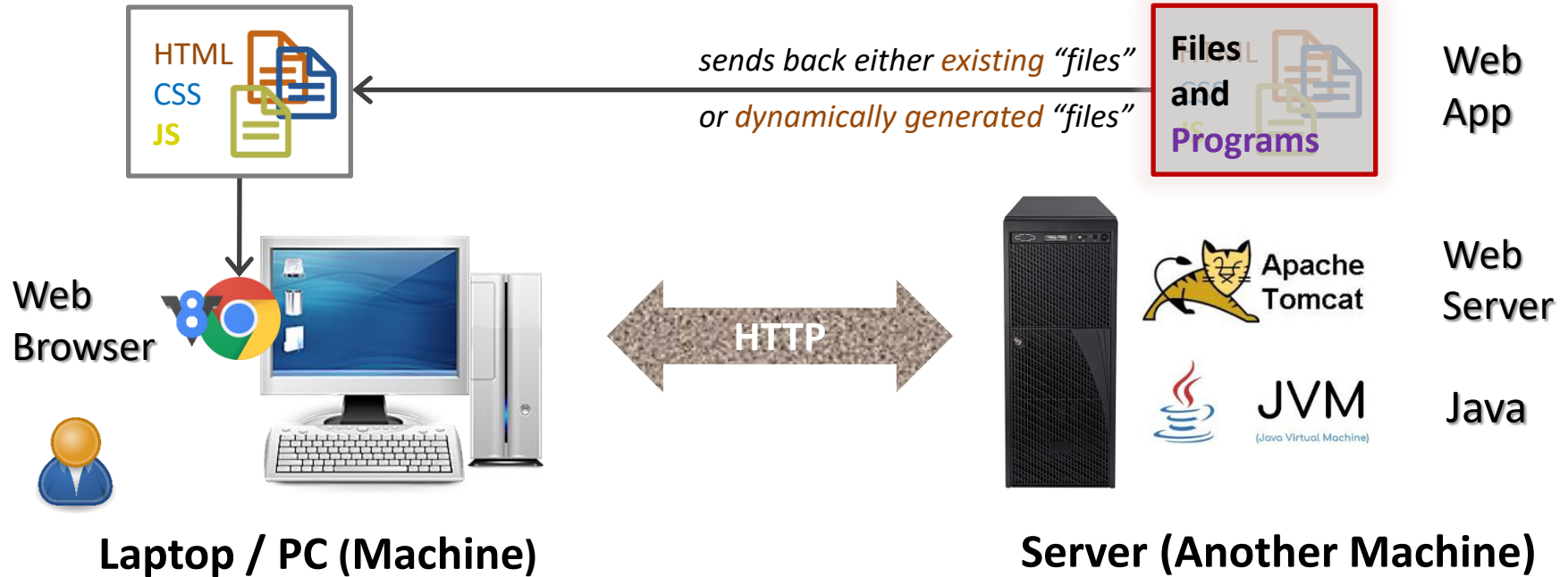
Lectures & Labs

April 2026							May 2026						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
			1	2	3	4						1	2
5	6	7	8	9	10	11	3	4	5	6	7	8	9
12	13	14	15	16	17	18	10	11	12	13	14	15	16
19	20	21	22	23	24	25	17	18	19	20	21	22	23
26	27	28	29	30			24	25	26	27	28	29	30
							31						

Client – Server Model

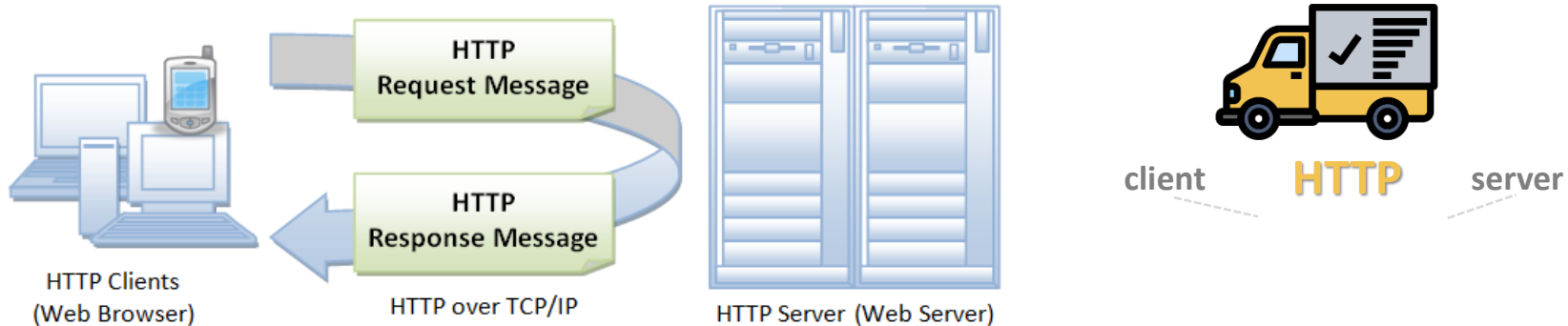
Client-side Code (aka Front-End)

Server-side Code (aka Back-End)



Data Transfer among Client & Server

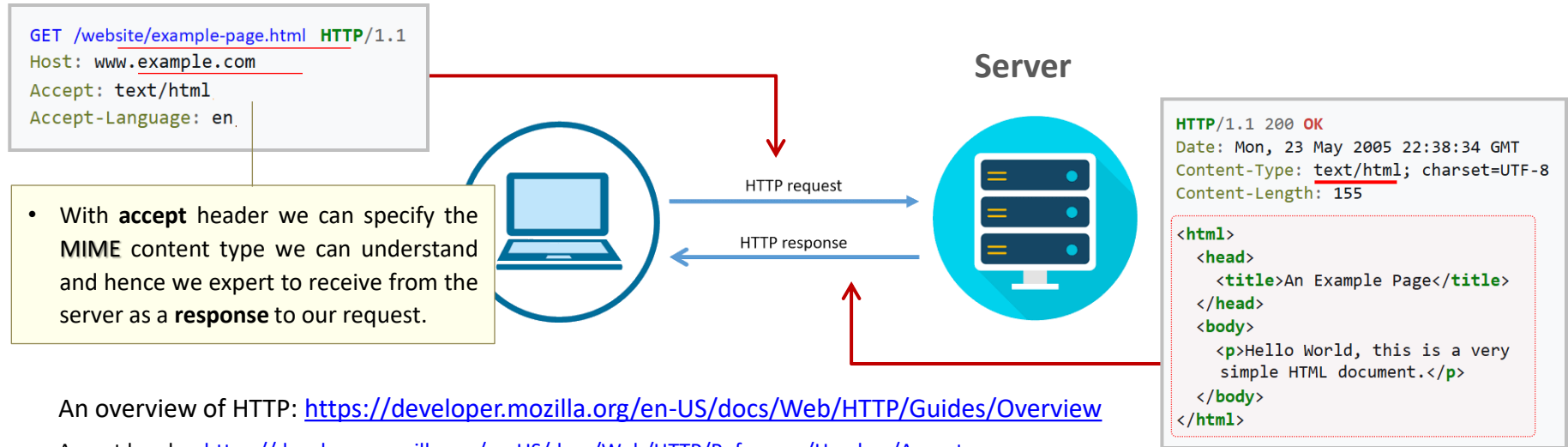
- [HTTP](#) and other Network Protocols of TCP/IP stack
 - It is being used to **transfer messages** between **HTTP Client** (e.g., Web Browser) and **HTTP Server** (e.g., Apache Tomcat)



- Two type of HTTP messages: HTTP Request and HTTP Response
 - The **payload** is often placed in the HTTP request/response **entity body**.

HTTP GET Request & Response

- Uses requests for a Web Page using their Web Browser
 - Web site **URL** is provided (by the User in the Web Browser's Address Bar)

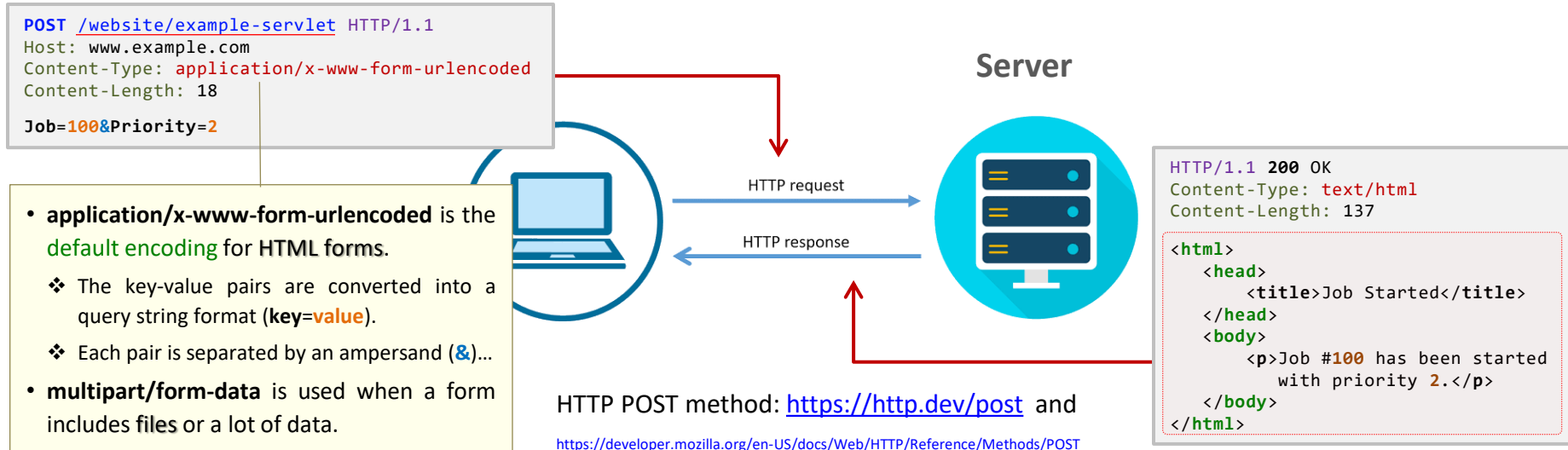


An overview of HTTP: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>

Accept header: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Accept>

HTTP POST Request & Response

- Uses submits an HTML Form using their Web Browser
 - A **Servlet** is invoked and relevant data are provided (by the Web Browser)



Common MIME Types

Kind of Document	MIME Type
HyperText Markup Language (HTML)	text/ html
Cascading Style Sheets (CSS)	text/ css
JavaScript (JS)	text/ javascript
Portable Network Graphics (PNG) image	image/png
Joint Photographic Experts Group (JPEG) image	image/jpeg
Extensible Markup Language (XML)	application/ xml
JavaScript Object Notation (JSON)	application/ json

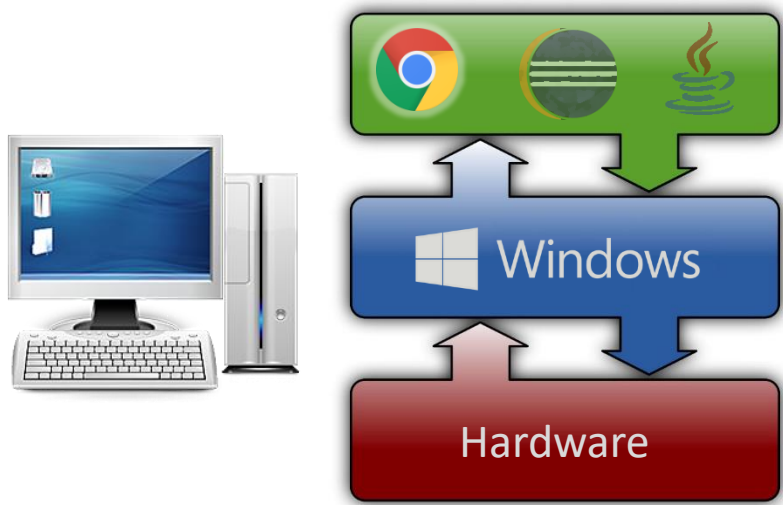
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/MIME_types/Common_types

Data Exchange between Client & Server

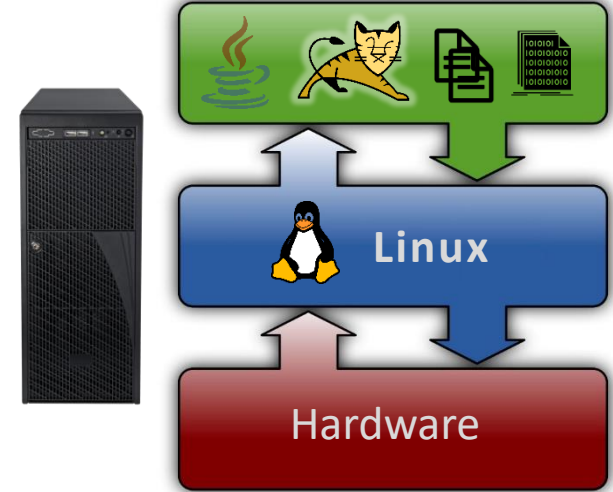
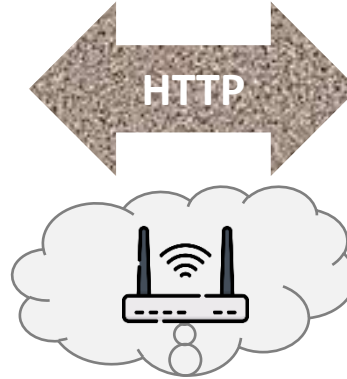
- Ask for an existing **File** (e.g., the main page of a Web Application)
 - HTTP Request
 - The Web Page **URL** (specified in the **1st line** of an HTTP request)
 - HTTP Response
 - **The Content of the File** (e.g., HTML code, provided in the HTTP **entity body**)
- Ask for the execution of a **Program** (e.g., data storage in DB)
 - HTTP Request
 - The Web Program **URL** (specified in the **1st line** of an HTTP request)
 - Given **data** provided (e.g., in the **1st line of HTTP request** or **entity body**)
 - HTTP Response
 - **The output** from the execution of the program (e.g., HTML code in the ...)

Client Machine vs Server Machine

User 

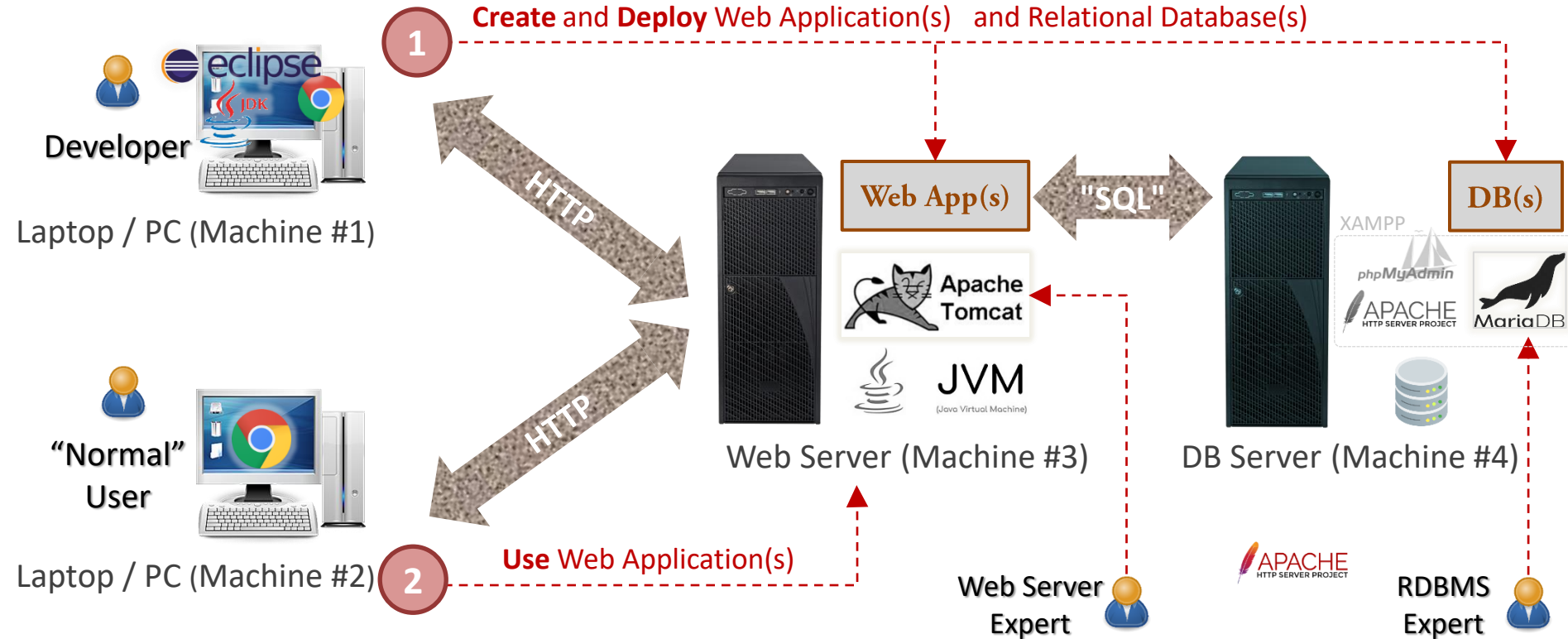


Laptop / PC (Machine)

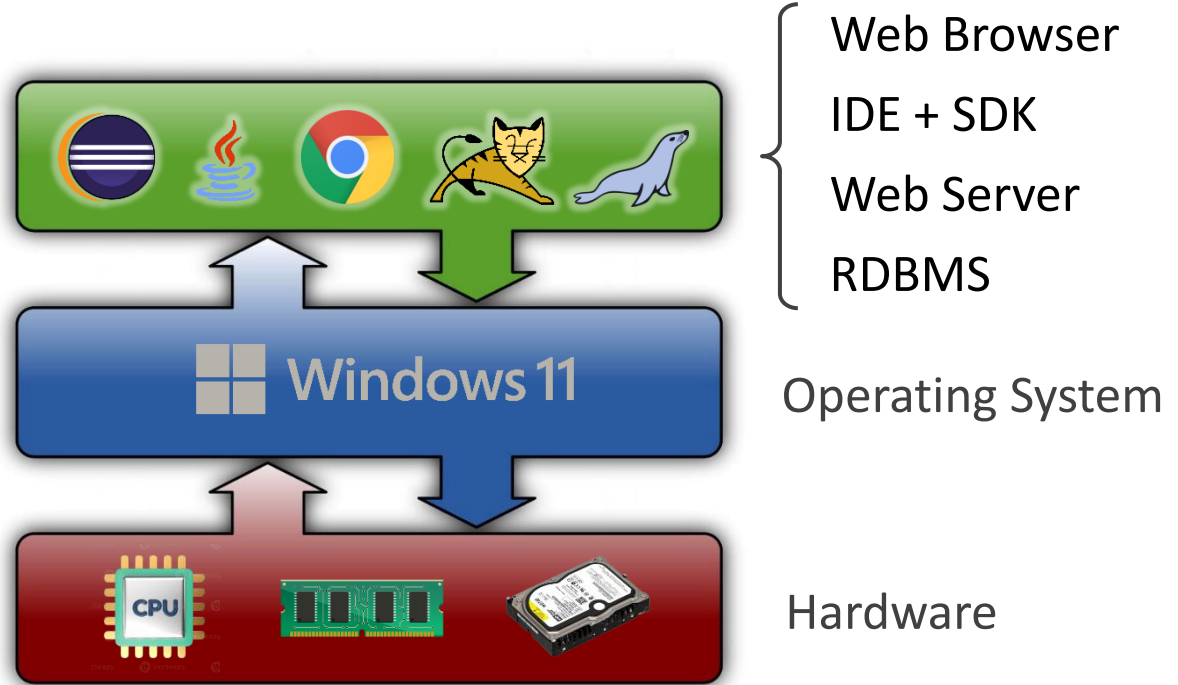


Server (Machine)

Users, Machines and Software Programs



Development and Testing Environment



Internet Applications

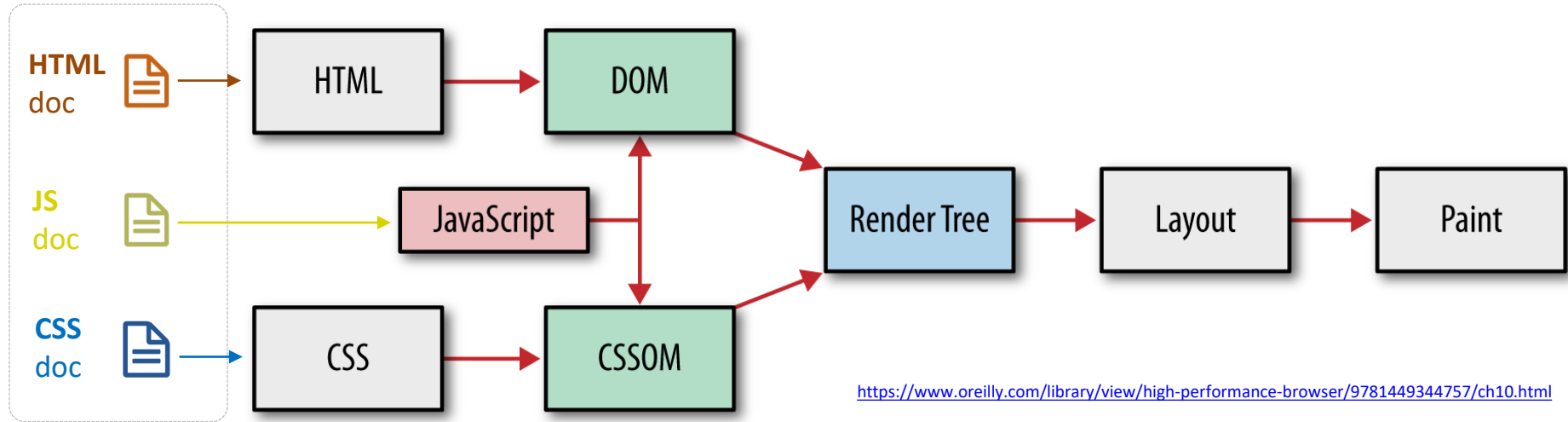
Client-side Programming

- Client-side programming involves **programs** that **run on a client's or user's device**.
- Client-side developers focus on creating **the part of a website** with which **the user interacts**.
 - Designing user interfaces (UI)
 - Creating website layouts
 - Adding visual design elements like colors and fonts
 - Adding form validation
 - Making website features more functional



Web Browser

- The web browser **retrieves** relevant documents from a web application server (using HTTP) and then **displays** the data on the user's screen.

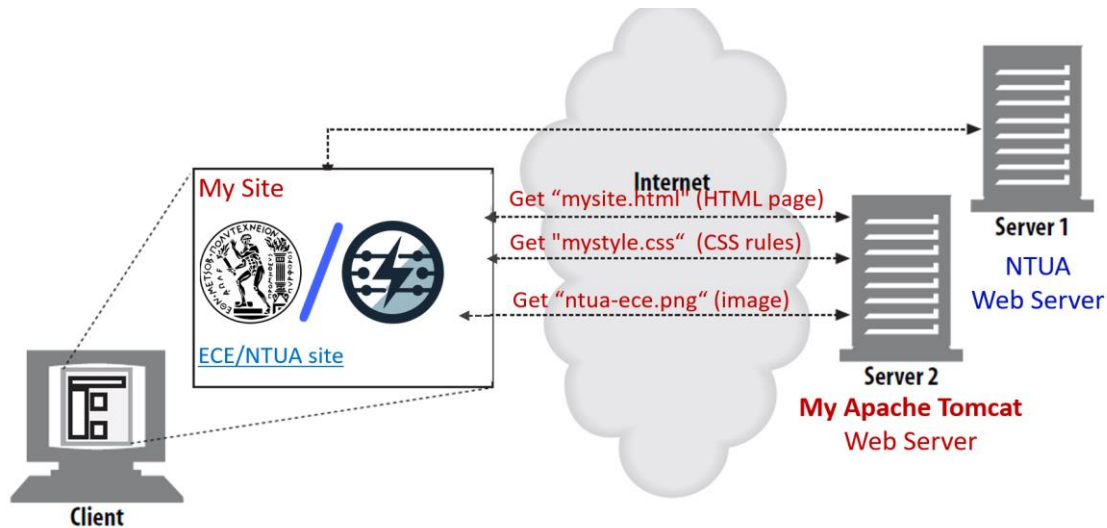


- It uses a **JavaScript Engine** to **compile** and **execute** JavaScript source code, Just-In-Time (JIT) compilation

Web Pages & HTTP Transactions

- **HTTP Transactions (i.e., Request and Response)**
 - For each Web Page several HTTP Transactions may take place

```
<!DOCTYPE html>
<html>
  <head>
    <title>Test Web Page</title>
    <link rel="stylesheet" href="style.css">
  </head>
  <body>
    <h1>My Site</h1>
    
    <a href="https://www.ece.ntua.gr/">
      ECE/NTUA site</a>
  </body>
</html>
```



HTML – CSS – JavaScript (1/2)

index.html

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Show Me</title>
6     <link href="page.css" rel="stylesheet">
7   </head>
8   <body>
9     <h1>A List presented on click..</h1>
10    <div id="divid">Press me !</div>
11    <ul id="ulid">
12      <li>Element 1</li>
13      <li>Element 2</li>
14      <li>Element 3</li>
15    </ul>
16    <script src="page.js"></script>
17  </body>
18 </html>
```



page.css

```
1 * {
2   font-family: Verdana, Geneva, Tahoma, sans-serif;
3 }
4
5 h1 {
6   color: darkblue;
7 }
8
9 #divid {
10   color: darkred;
11   background-color: lightgray;
12   border: 1px solid darkgray;
13   border-radius: 10px;
14   margin-left: 20px;
15   padding: 5px 10px;
16   width: 200px;
17   text-align: center;
18 }
19
20 #divid:hover {
21   cursor: pointer;
22   background-color: lightgreen;
23 }
```



HTML – CSS – JavaScript (2/2)

index.html

HTML



```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <meta charset="UTF-8">
5     <title>Show Me</title>
6     <link href="page.css" rel="stylesheet">
7   </head>
8   <body>
9     <h1>A List presented on click.</h1>
10    <div id="divid">Press me !</div>
11    <ul id="ulid">
12      <li>Element 1</li>
13      <li>Element 2</li>
14      <li>Element 3</li>
15    </ul>
16    <script src="page.js"></script>
17  </body>
18 </html>
```

page.js

```
1
2
3 // DIV Element
4 let div = document.querySelector("#divid");
5 div.addEventListener("click", function(e) {
6   // Show or Hide UL element
7   let ul = document.querySelector("#ulid");
8   if (ul.style.display !== "none") {
9     ul.style.display = "none";
10  } else {
11    ul.style.display = "block";
12  }
13 });
14
15
```

JavaScript



Server-side Programming

- Server-side programming refer to the development of software **programs** that are **run on a server** (aka **backend**)
- Server-side programming languages:



Java

Servlet and JSP

Java Servlet & JSP pages

- **Servlet**
 - Servlets are small **programs** (i.e., bytecode) running on the server side.
 - Servlet runs on a **Java-enabled** web application server.
 - Users often interact with Servlets using **HTTP** protocol
- **JSP**
 - A JSP page consists of **HTML code** and **JSP elements** (i.e., Java ...)
 - e.g., a **Scriptlet** is a block of **Java code** enclosed between **<%** and **%>**
 - A JSP page is automatically converted it into a **Java Servlet**

Hello Servlet World !

HelloServlet.java

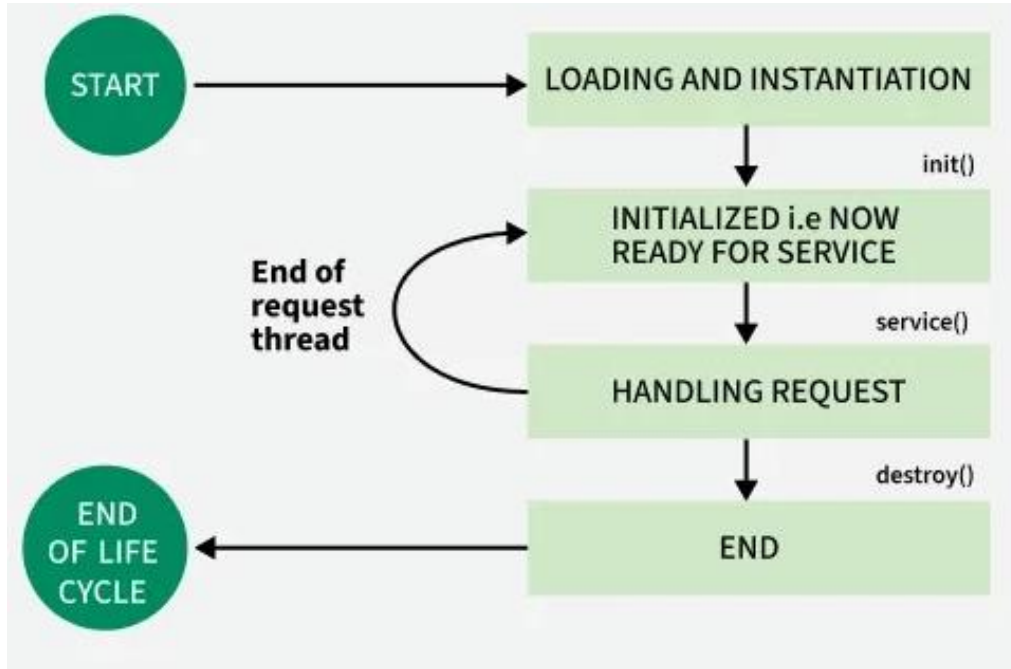
```
import java.io.*;
import jakarta.servlet.*;

public class HelloServlet extends GenericServlet {

    public void service(ServletRequest request, ServletResponse response)
    throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>Hello!");
        pw.close();
    }
}
```

All the classes and/or
interfaces mentioned
here can be found in
the **servlet-api.jar** file

Servlet Life Cycle



- **init():** It is invoked only when the servlet is first loaded into memory.
- **service():** It is called for each HTTP request.
- **destroy():** It is called when the servlet is being removed from memory.

The above methods are specified in the **Servlet** Interface.

<https://medium.com/@hpdeshmukh/understanding-the-servlet-life-cycle-and-types-of-servlets-in-java-web-development-d433a8fe21e7>

HttpServlet

- The **HttpServlet** is an **Abstract** class that extends **GenericServlet**.

Method	Description
<code>void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException</code>	Handles an HTTP GET request.
<code>void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException</code>	Handles an HTTP POST request.
.....	
<code>void service(HttpServletRequest req, HttpServletResponse res) throws IOException, ServletException</code>	Called by the server when an HTTP request arrives for this servlet. The arguments provide access to the HTTP request and response, respectively.

Http Servlet Request and Response

- The **HttpServletRequest** interface enables a servlet to **obtain information** about a client request.

<code>String getParameter(String <i>pname</i>)</code>	Returns the value of the parameter named <i>pname</i> .
<code>Enumeration<String> getParameterNames()</code>	Returns an enumeration of the parameter names for this request.
<code>Cookie[] getCookies()</code>	Returns an array of the cookies in this request.

- The **HttpServletResponse** interface enables a servlet to **formulate an HTTP response** to a client.

<code>void setContentType(String <i>type</i>)</code>	Sets the content type for the response to <i>type</i> .
<code>PrintWriter getWriter()</code> throws <code>IOException</code>	Returns a PrintWriter that can be used to write character data to the response.
<code>void addCookie(Cookie <i>cookie</i>)</code>	Adds <i>cookie</i> to the HTTP response.
<code>void setStatus(int <i>code</i>)</code>	Sets the status code for this response to <i>code</i> .

Servlet Example

ColorPostServlet.java

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class ColorPostServlet extends HttpServlet {

    public void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String color = request.getParameter("color");

        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.println("<B>The selected color is: ");
        pw.println(color);
        pw.close();

    }

}
```



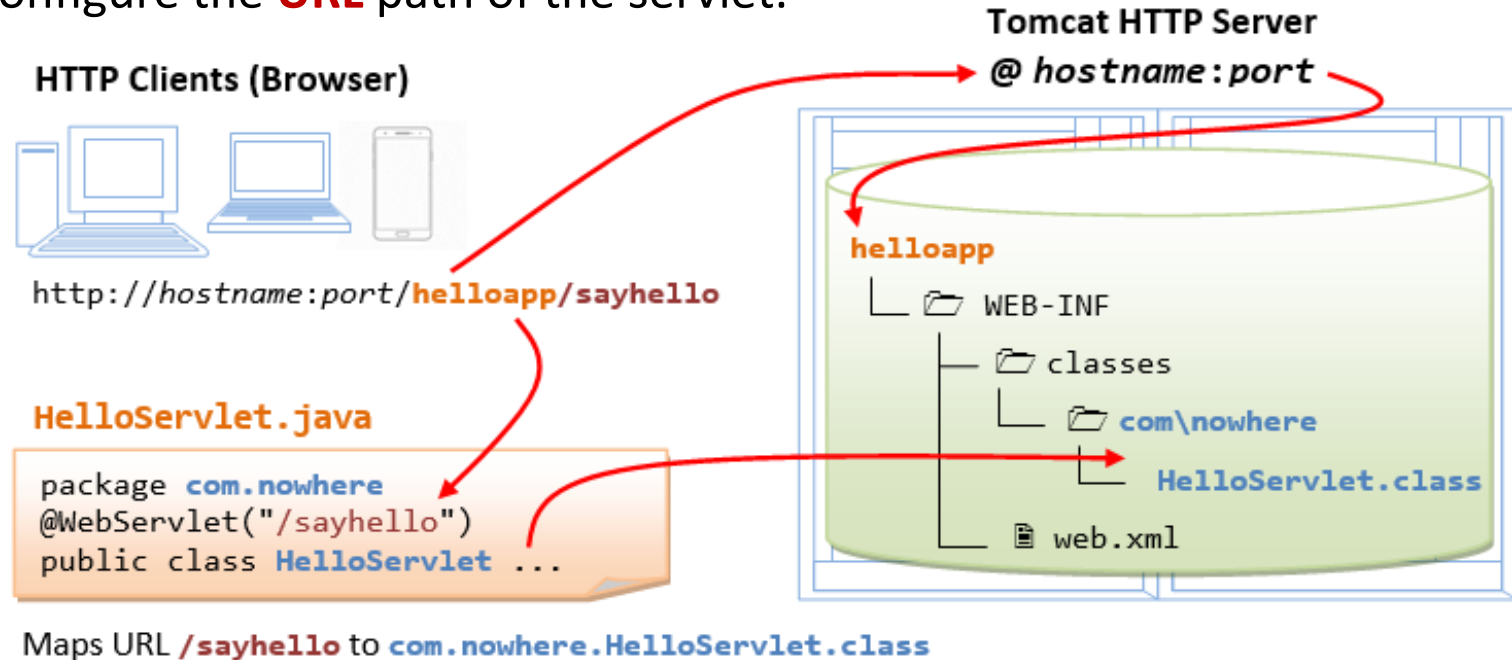
The response is
an **HTML** «file»



It may also contain **CSS & JavaScript**

Servlet Path using Annotations

- Starting from Servlet API 3.0, we can use annotation **@WebServlet** to configure the **URL** path of the servlet.



HTML & Java Servlet – Example Code

```
@WebServlet("/TestServlet")
public class TestServlet extends HttpServlet {
    // Data
    private final Map<Integer, UserData> map = new HashMap<>();
    public TestServlet() {
        super();
        map.put(1, new UserData("John", "male", 25));
        map.put(2, new UserData("Maria", "femail", 23));
        map.put(3, new UserData("Eleni", "female", 30));
    }
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
        // Input
        final String uidstr = request.getParameter("uid");
        final Integer uid = Integer.parseInt(uidstr);
        // Process
        final UserData ud = map.get(uid);
        // Output
        response.setContentType("text/html");
        PrintWriter pw = response.getWriter();
        pw.append("Message from Server ... ");
        pw.append("'" + ud);
    }
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        doGet(request, response);
    }
}
```

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>home</title>
  </head>
  <body>
    <h1>Form</h1>
    <form action="TestServlet" method="GET">
      <input type="text" name="uid">
      <input type="submit" value="Call Servlet">
    </form>
  </body>
</html>
```

Hello JSP World !

hello.jsp

```
<%@page language="java" contentType="text/html" %> ← It's a JSP
```

```
<html>
```

```
  <head>
```

```
    <title>Hello World dynamic HTML</title>
```

```
  </head>
```

```
  <body>
```

```
    Hello World!
```

```
    <%
```

```
      String userAgent = request.getHeader("user-agent");
```

```
      out.println("<br/>user-agent " + userAgent);
```

```
    %>
```

```
  </body>
```

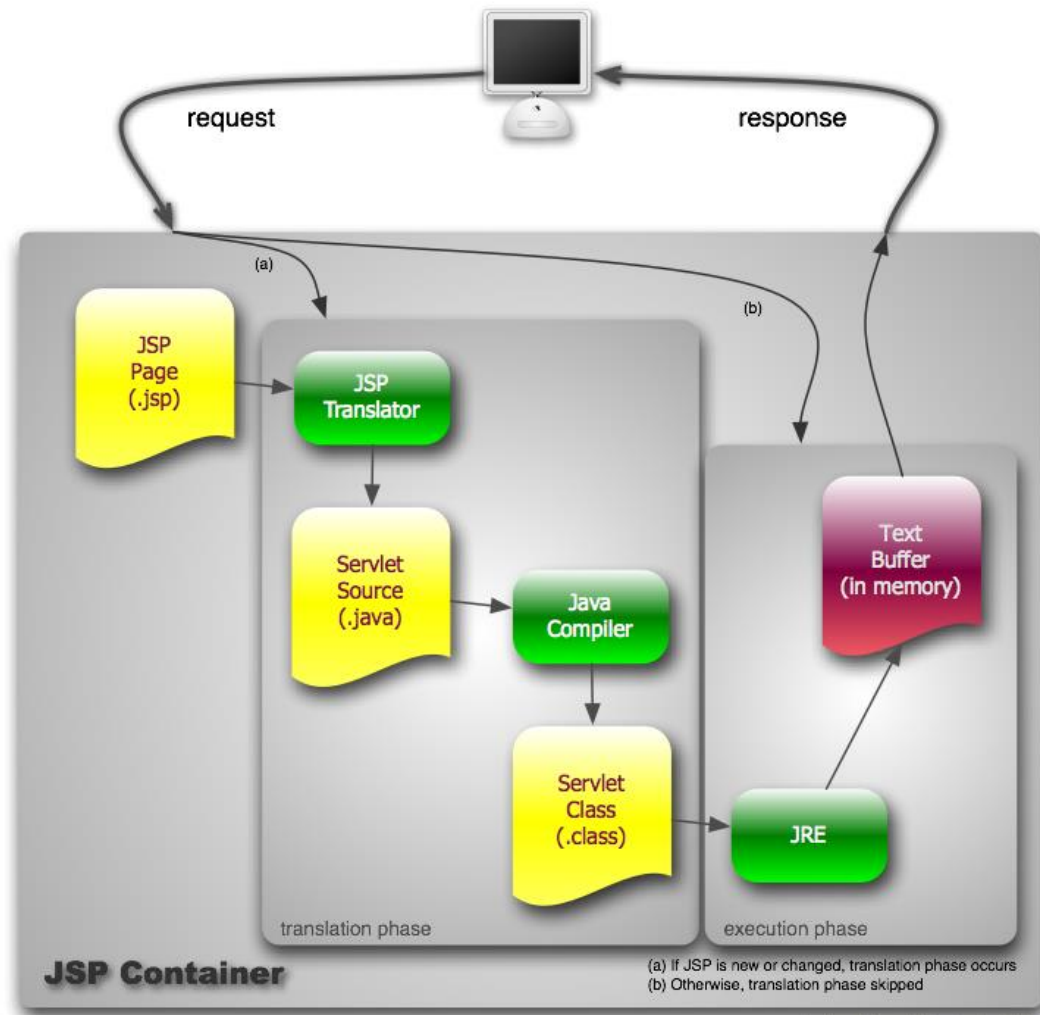
```
</html>
```

**Java
Code**

JSP Life Cycle

- The **first time** that your JSP is **requested** the **translation phase** will be triggered.
- Translation phase will also be automatically triggered if the **JSP has been changed** since the last time it was translated.
- Then **servlet class**, loaded and it is ready to run in the **execution phase**.

<https://javaranch.com/journal/200510/secret-life.html>



JSP Elements

- A **JSP page** is made out of a page template, which consists of **HTML code** and **JSP elements**.
 - **Scripting Elements:** java fragments, capable of manipulating **Java** objects, invoking their methods and catching Java exceptions.

```
<%= new java.util.Date() %>
```
 - **Directive Elements:** provide information on the page itself, necessary for its translation.

```
<%@page import="java.util.Date" %>
```
 - **Action Elements:** activities that, like the scripting elements, need to be performed when the page is requested.

Scripting Elements

- Scripting elements let you embed Java code in an HTML page.

- **Scriptlet**: a block of Java **code** enclosed between `<%` and `%>`

```
<% if (condition) { %>  
    <p>This is only shown if the condition is satisfied</p>  
<% } %>
```

- **Expression**: the **result** of a Java expression enclosed in the pair `<%=` and `%>`

```
<%@page import="java.util.Date"%>  
Server date and time: <%=new Date()%>
```

- **Declaration**: a Java **variable** declaration enclosed between `<%!` and `%>`

HTML and JSP – Example Code

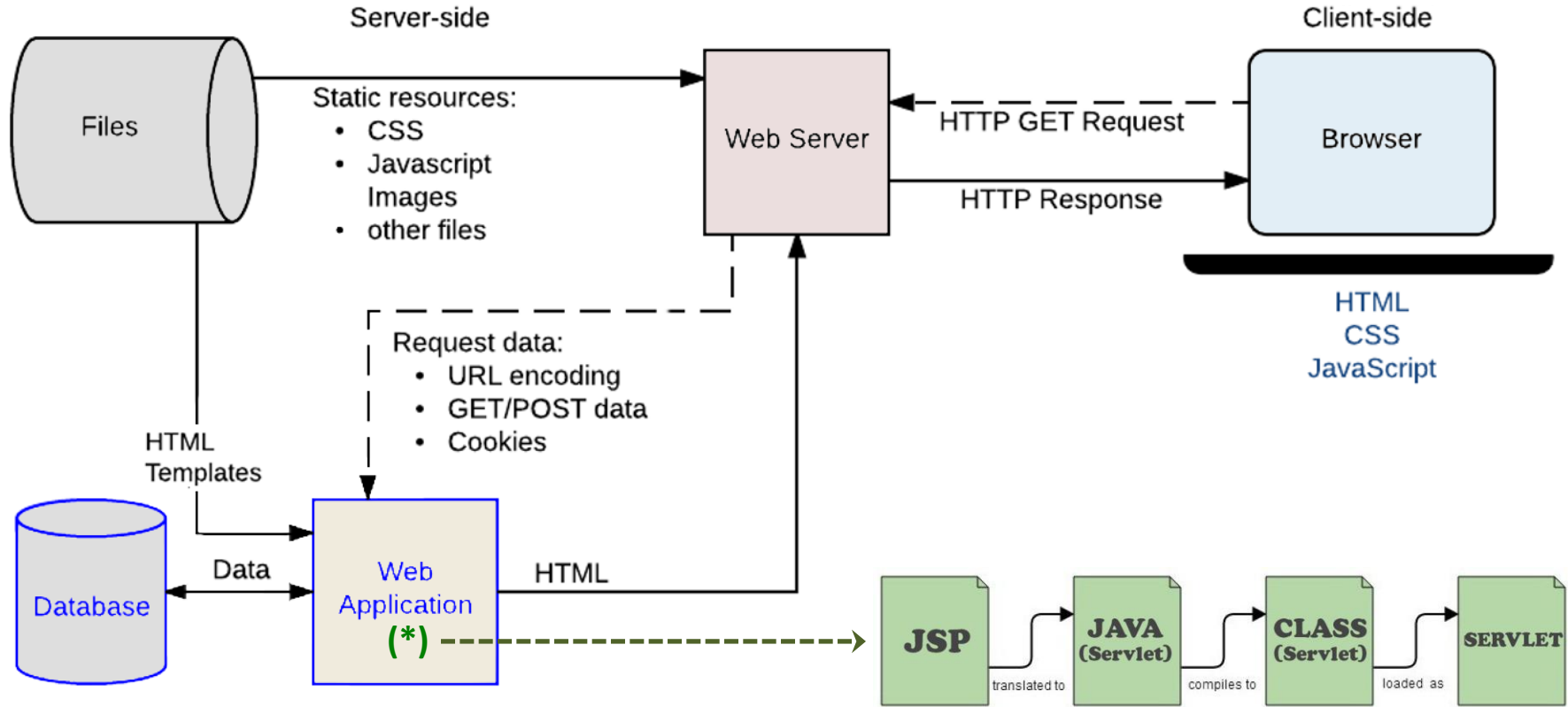
```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<%@page import="java.util.Map"%>
<%@page import="java.util.HashMap"%>
<%@page import="test.UserData"%>
<!DOCTYPE html>
<html>
  <head> <meta charset="UTF-8"> <title>Test JSP</title> </head>
  <body>
    <!-- Map<Integer, UserData> map = new HashMap<>(); -->
    <%
      map.put(1, new UserData("John", "male", 25));
      map.put(2, new UserData("Maria", "female", 23));
      map.put(3, new UserData("Eleni", "female", 30));
    %>
    <h1>Table</h1>
    <table border="1">
      <tr> <th>Name</th> <th>Sex</th> <th>Age</th> </tr>
      <% for (UserData ud : map.values()) { %>
        <tr> <td><%=ud.getUsername()%></td> <td><%=ud.getSex()%></td> <td><%=ud.getAge()%></td> </tr>
      <% } %>
    </table>
  </body>
</html>
```

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8">
    <title>home</title>
  </head>
  <body>
    <h1>Form</h1>
    <form action="TestServlet" method="GET">
      <input type="text" name="uid">
      <input type="submit" value="Call Servlet">
    </form>

    <h1>Link</h1>
    <a href="TestJSP.jsp">Call JSP</a>

  </body>
</html>
```


Interaction between Client & Server



Apache Tomcat Server and Eclipse IDE

Apache Tomcat Server

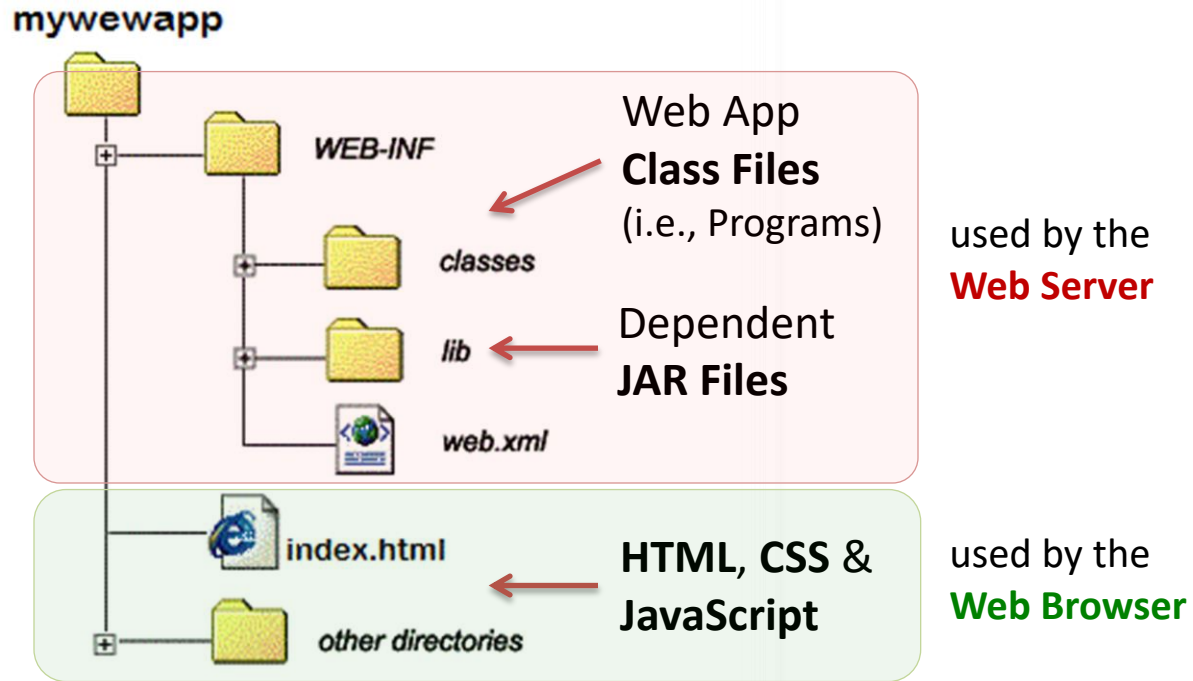
- Apache Tomcat is a free, open-source **Web Server** and **Java Servlet container**.
 - Web Site: <https://tomcat.apache.org/>
- Download **Tomcat 11** core ZIP & Extract files to Folder – Run: bin/startup.bat 
 - Tomcat 11.0 requires **Java 17** or later.
 - Server URL: <http://localhost:8080/>
- Tomcat Server Directories and Files
 - **/bin**: Startup and Shutdown scripts
 - **/conf**: Configuration files (e.g., server.xml, tomcat-users.xml)
 - **/logs**: Log files are here by default
 - **/webapps**: This is where your web apps go
 - **WAR** files are placed in this folder

The **java version** being used is specified in these scripts

The Apache Tomcat Server administrator **username** and **password** can be specified here

Web Application Resources

- Web Archive (WAR)
 - An **Archive** that contains all the files of a web application
 - A ZIP file with specific directories for Java Class Files

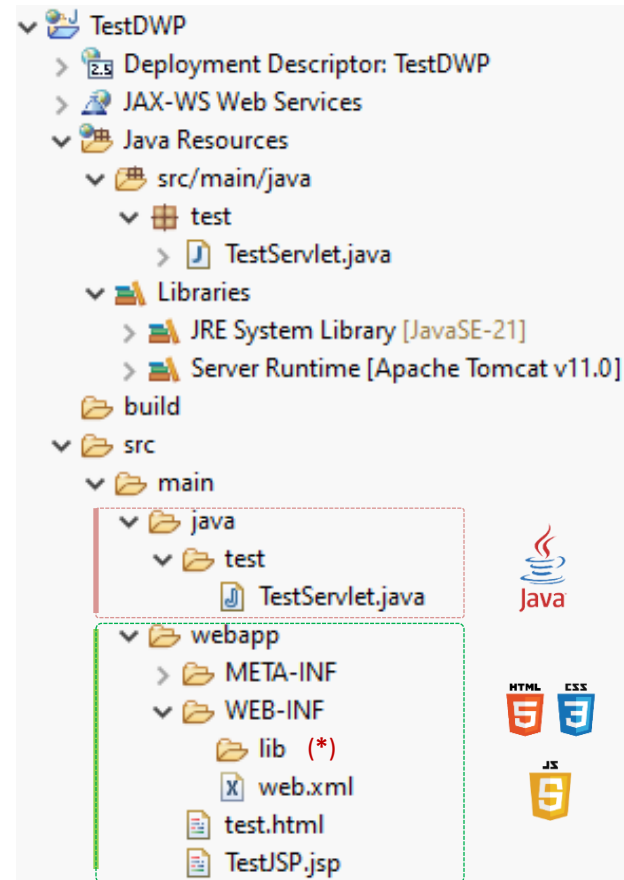


Java Web Application: https://www.javawebtutor.com/articles/servlets/structure_of_java_web_application.php

Java IDE – Web App – Source Files

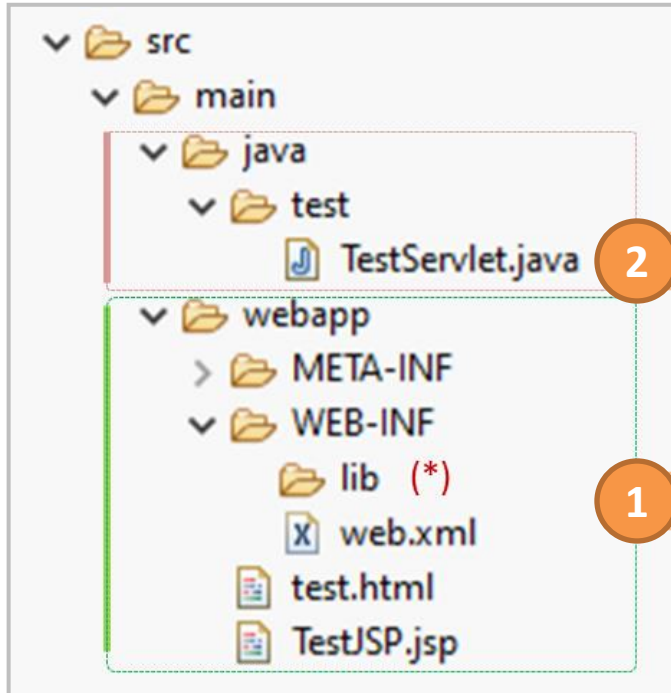
Using an **IDE** (for Java) such as **Eclipse** (Dynamic Web Project)

- HTML, CSS & JavaScript Files and JSP pages
 - residing in the "**webapp**" Folder
- Java Servlet and Other Java Files
 - residing in the "**java**" folder (Java Resources)
- External Java Libraries (i.e., JAR files)
 - can be placed in the "**webapp/WEB-INF/lib**" folder

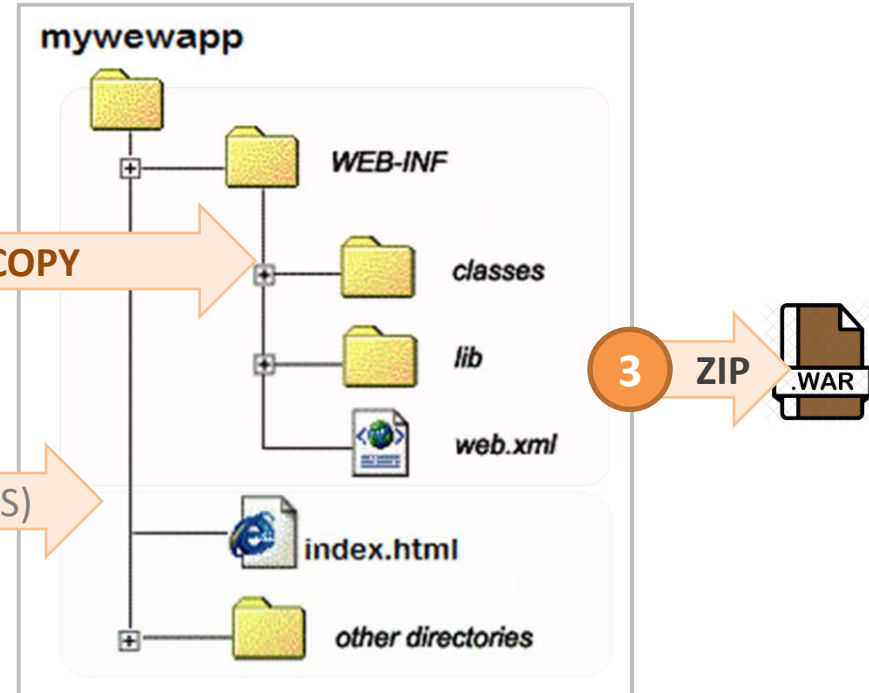


Manually Create a WAR File

- What we already have:



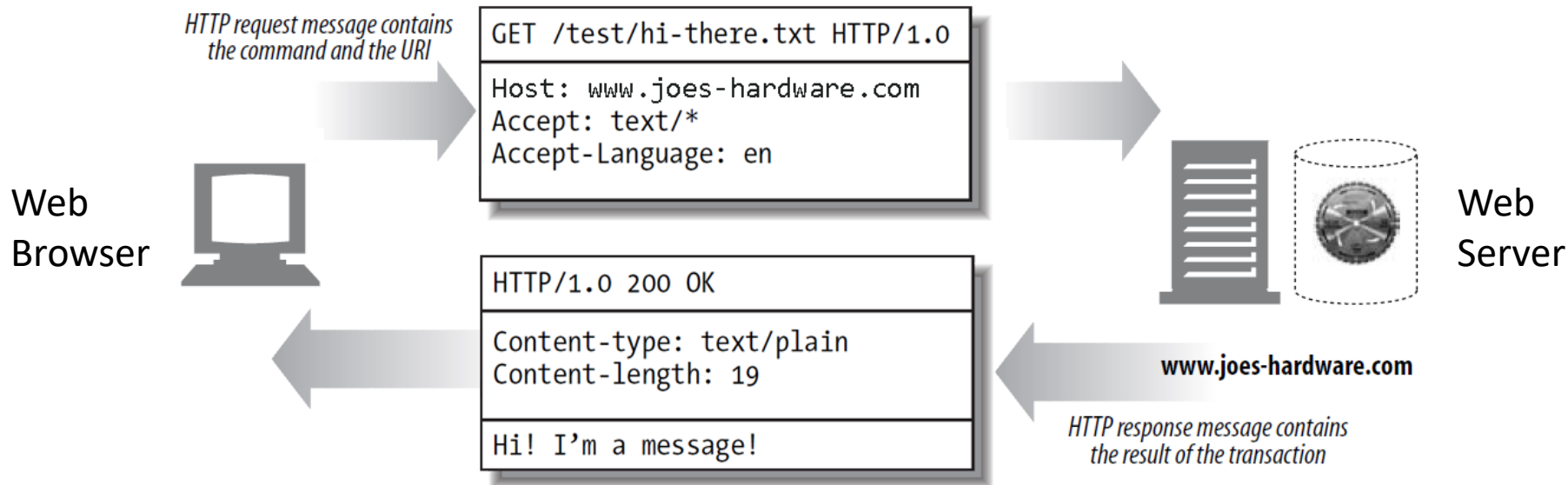
- What we want:



HTTP

Sessions and Cookies

HTTP



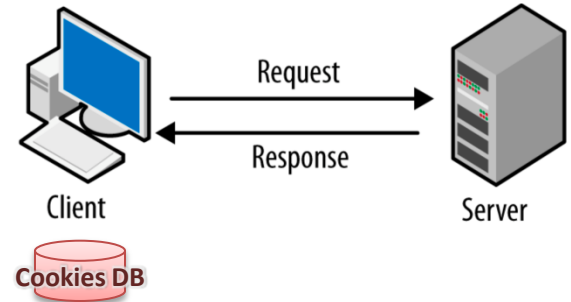
HTTP is a **stateless** protocol:

- Each transaction (i.e., request/response) happens **in isolation**.

Cookies

- **Cookies** enable web applications to **identify users** across HTTP transactions.

Basic idea: The **web browser** accumulates a set of **server-specific** information and **provide this information back to web server** each time we request for another web resource.



- Type of Cookies:
 - **Session Cookies** (stored *in memory*) **deleted** on browser exist.
 - **Persistent Cookies** (stored *on disk*) **survive** browser exits.

In both cases, the **cookies** (from all sites) are stored (either in memory or on disk) on the **client-side**.

(a)

```
GET /index.html HTTP/1.0
Host: www.joes-hardware.com
```

The **web client** makes an **HTTP request** for a specific web resource (i.e., URL) for the very first time.



Client

Internet



Server

(b)

```
HTTP/1.0 200 OK
Set-cookie: id="34294"; domain="joes-hardware.com"
Content-type: text/html
Content-length: 1903
...
```

Set-Cookie

id=34294

The **web server** "adds" a **cookie** (i.e., name=value) in **HTTP response** sent back to web client.

(*) The **cookie** is locally stored by the web client !



Web Browser
Cookies DB



Client

Internet



Server

(c)

```
GET /index.html HTTP/1.0
Host: www.joes-hardware.com
Cookie: id="34294"
```

Cookie

id=34294

The **web client** makes another **HTTP request** for (another) web resource, and provides **relevant cookie**.



Client

Internet



Server

Web Server – HTTP response

- When a **web server** receives an HTTP request, it can *optionally* add in the HTTP response header the **Set-Cookie** field, to instruct the **web browser** to store the data (i.e., **cookie-name=cookie-value**)
 - Multiple Set-Cookie headers can be used to store more than one piece of data
- Cookies can also include certain **prefixes** and **attributes** that further specify how they can be used and how long they last.

HTTP
Response
Header

```
...  
Set-Cookie: name=value [; Expires=date] [; Domain=domain] [; Path=path] [; Secure] [; HttpOnly]  
...
```

❖ **Note:** Permanent cookies have either a **Max-Age** or **Expires** attribute.

Using HTTP cookies: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Cookies>

Web Browser – HTTP Request

- The **web browser** decides which cookies should be sent to which web server. In general, a web browser sends to a web server **only those cookies** that the web server generated.
 - The web browser adds in the HTTP request header a **Cookie** field, with all the relevant cookies (including both their name and their value).

HTTP
Request
Header

```
...  
Cookie: name1=value1 [; name2=value2] [; name3=value3] ...  
...
```

- The **web server** generating a cookie **can control** which sites get to see that cookie by adding a **Domain** attribute. We can further restrict access to a cookie using a **Path** attribute.

Sessions

- Sessions are a way of storing information about a user on the **server-side**.
 - When a user logs into the website, a session is created (by the **web server**), in which you can store data in a **key/value** format (like cookies).
 - This session is associated with a randomly generated unique ID, called “**session ID**”.
 - The generated session ID is then sent to the user’s browser, using a **cookie** !
 - When the **browser** send a request to the web server, it’ll send the cookie with the **session-id** which is used for locating session data.



Sessions are often used to store **sensitive information** such as user credentials and financial data. They are more secure than cookies because the information is stored on the server-side...

How session and cookies works: <https://medium.com/@hendelRamzy/how-session-and-cookies-works-640fb3f349d1>

Sessions vs Cookies

Feature	Cookies	Sessions
Storage Location	Stored on the client side (browser).	Stored on the server side.
Data Security	Less secure, as it is exposed to the client.	More secure, as data is stored on the server.
Performance	Faster as data is stored on the client.	Slightly slower as each request requires server processing.
Data Size Limit	Limited to 4KB per cookie.	Can store large amounts of data.
Expiration	Can be set manually (maxAge, expires).	Expires automatically after inactivity or when explicitly destroyed.
Data Persistence	Persists even after the browser is closed (unless expired).	Data is lost once the session expires or the server restarts (unless stored in a database).
Used for Authentication	Often used for storing authentication tokens	Commonly used for session-based authentication.
Example	Google, Facebook, Amazon, YouTube, Netflix etc.	Banking Websites, E-learning Platforms, Job Portals etc.

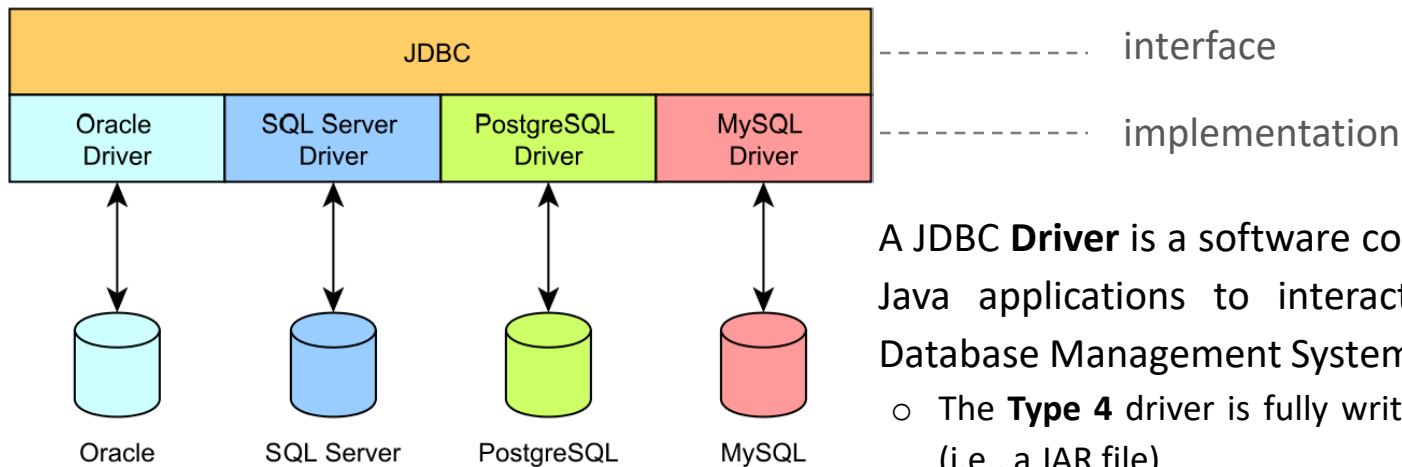
Difference Between Session and Cookies: <https://www.geeksforgeeks.org/difference-between-session-and-cookies/>

JDBC

(Java Database Connectivity)

JDBC

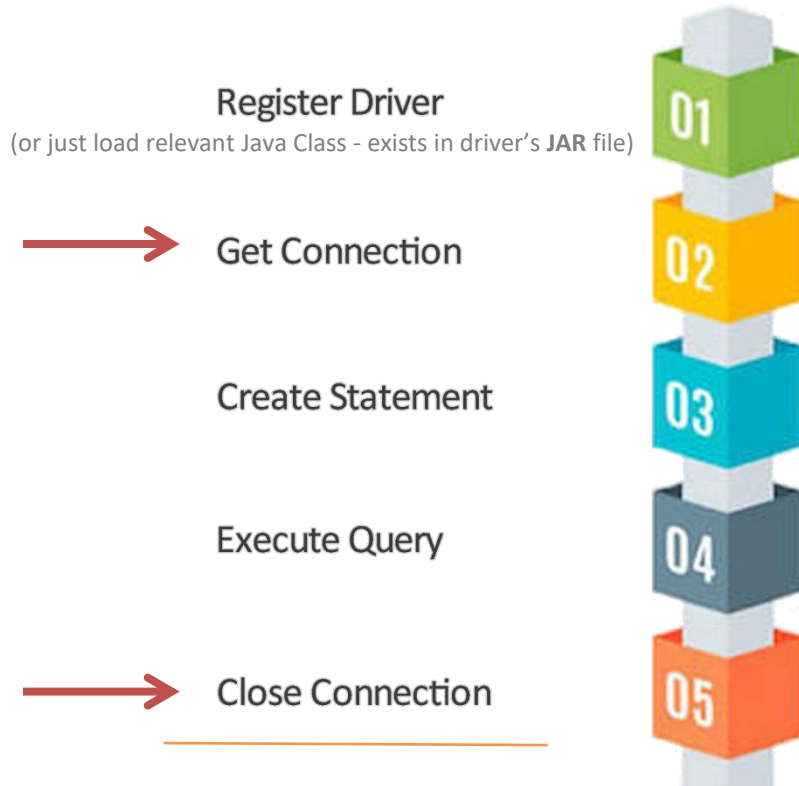
- **JDBC** stands for **J**ava **D**atabase **C**onnectivity
- It is an Application Programming Interface (**API**) that helps Java applications to communicate with a database server (aka **DBMS**).



A JDBC **Driver** is a software component enabling Java applications to interact with a specific Database Management System (DBMS).

- The **Type 4** driver is fully written in Java language (i.e., a JAR file)

Interaction with a Database



- **JDBC API Interfaces (`java.sql.*`)**

- **Connection**
- **Statement / PreparedStatement**
 - SQL command
- **ResultSet**
 - SELECT SQL query execution results (table of data)

❖ Ensure that Statement (and ResultSet) are closed

An Example

- **Insert** data provided in the following SQL table of a DB
 - Assumption: We already know the **DB_URL**, **USER** and **PASSWORD**
 - Interaction with DB using SQL query: `INSERT INTO my_table VALUES (?, ?, ?, ?)`

```
final String SQL = "INSERT INTO my_table VALUES (null, ?, ?, ?)";
```

```
Connection conn = DriverManager.getConnection(URL, USER, PASS);
```

```
PreparedStatement pst = conn.prepareStatement(SQL);
```

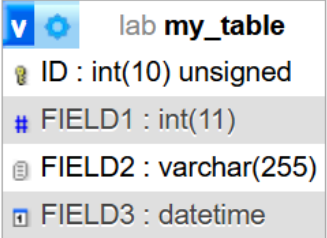
```
pst.setInt(1, 123);
```

```
pst.setString(2, "string value");
```

```
pst.setDate(3, null);
```

```
int response = pst.executeUpdate();
```

```
System.out.println("Rows affected: " + response);
```



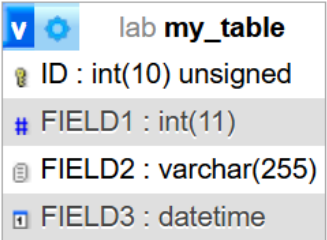
	lab my_table
ID :	int(10) unsigned
# FIELD1 :	int(11)
FIELD2 :	varchar(255)
FIELD3 :	datetime

DB
table

Another Example

- **Present** in the standard output all the rows of an SQL table of a DB
 - Assumption: We already know the **DB_URL**, **USER** and **PASSWORD**
 - Interaction with DB using SQL query: **SELECT * FROM table**

```
final String SQL = "SELECT * FROM my_table";
Connection conn = DriverManager.getConnection(URL, USER, PASS);
Statement st = conn.createStatement();
ResultSet rs = st.executeQuery(SQL);
while(rs.next()) {
    final Integer id = rs.getInt(1);
    final Integer int_value = rs.getInt(2);
    final String str_value = rs.getString(3);
    final Date dt_value = rs.getDate("FIELD3");
    System.out.println(id + "-" + int_value + "-" + str_value + "-" + dt_value);
}
```

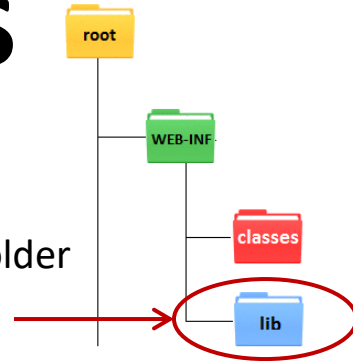


lab my_table	
ID	int(10) unsigned
# FIELD1	int(11)
# FIELD2	varchar(255)
# FIELD3	datetime

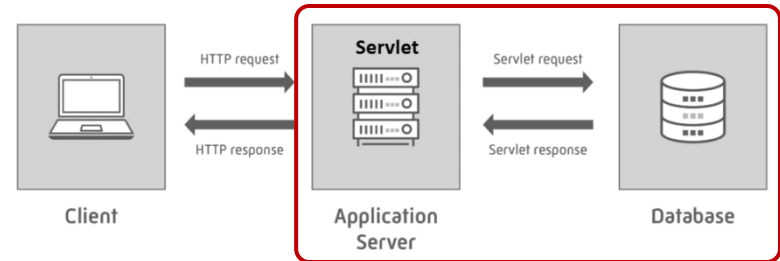
DB
table

Web Application and RDMS

- A Web Application that interacts with a Relational Database
 - The **Maria DB JDBC Driver JAR** file should be placed in the **WEB-INF/lib** folder
- Java Servlet – **doGet(..) / doPost(..)** method Java Code



- Open a new **Connection** to the RDMS (or re-use an existing one from a Connection Pool)
- Create a **Statement** and Execute it
- Handle the results
 - Get results using the **ResultSet** (SELECT SQL)
- **Release Resources** (e.g., close connection or return it back to the Connection Pool)



```
@WebServlet("/StoreData")
```

```
public class StoreData extends HttpServlet {
```

```
    private static final long serialVersionUID = 1L;
```

```
    private static final String USER = "root";
```

```
    private static final String PASS = "";
```

```
    private static final String URL= "jdbc:mariadb://localhost:3306/LAB";
```

```
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
```

```
        // Get HTTP Request Data (User Field Values)
```

```
        final Integer int_Value = Integer.parseInt(request.getParameter("field1"));
```

```
        final String str_value = request.getParameter("field2");
```

```
        final java.sql.Date date_value = getSqlDate(request.getParameter("field3"));
```

```
        // Store Data in the RDBMS - Insert a new Row in Table
```

```
        int execution_response = -1;
```

```
        try {
```

```
            final String SQL_1 = "INSERT INTO my_table VALUES (null, ?, ?, ?)";
```

```
            Connection conn = DriverManager.getConnection(URL, USER, PASS);
```

```
            PreparedStatement pst = conn.prepareStatement(SQL_1);
```

```
            pst.setInt(1, int_Value);
```

```
            pst.setString(2, str_value);
```

```
            pst.setDate(3, date_value);
```

```
            execution_response = pst.executeUpdate();
```

```
            pst.close();
```

```
            conn.close();
```

```
        } catch (Throwable t) {
```

```
            System.out.println("Problem - Cannot INSERT the given data !");
```

```
            t.printStackTrace();
```

```
        }
```

```
        // Prepare HTTP Response
```

```
        response.setContentType("text/html");
```

```
        PrintWriter pw = response.getWriter();
```

```
        pw.println("<html>");
```

```
        pw.println("<body>");
```

```
        pw.println("Rows affected: <strong>" + execution_response + "</strong> ");
```

```
        pw.println("</body>");
```

```
        pw.println("</html>");
```

```
    }
```

```
    public StoreData() {
```

```
        try {
```

```
            Class.forName("org.mariadb.jdbc.Driver");
```

```
        } catch (Throwable t) {
```

```
            throw new RuntimeException("Cannot find Maria DB JDBC driver", t);
```

```
        }
```

```
    }
```

```
    public java.sql.Date getSqlDate(final String str) {
```

```
        try {
```

```
            java.util.Date util_dt = new SimpleDateFormat("dd-MM-yyyy").parse(str);
```

```
            java.sql.Date sql_dt = new java.sql.Date(util_dt.getTime());
```

```
            return sql_dt;
```

```
        } catch (Throwable t) {
```

```
            throw new RuntimeException("String '" + str + "' to Date problem!", t);
```

```
        }
```

```
    }
```

Testing Servlet:

- Deploy the Web Application in the Apache Tomcat Server

➤ Create WAR and place it in the **webapps** folder

- Open a Web Browser and type the following URL

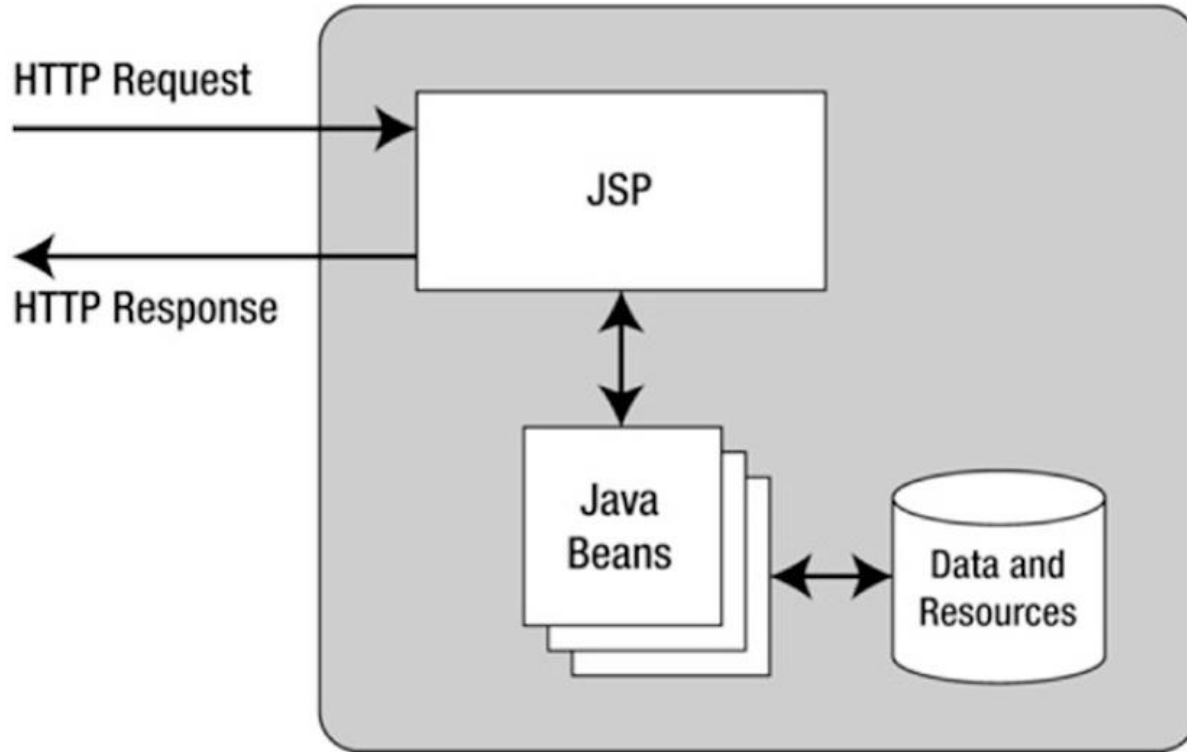
<http://localhost:8080/Lab3/StoreData?field1=234&field2=str&field3=05-02-2025>

```
    protected void doPost(HttpServletRequest request, HttpServletResponse response) {
```

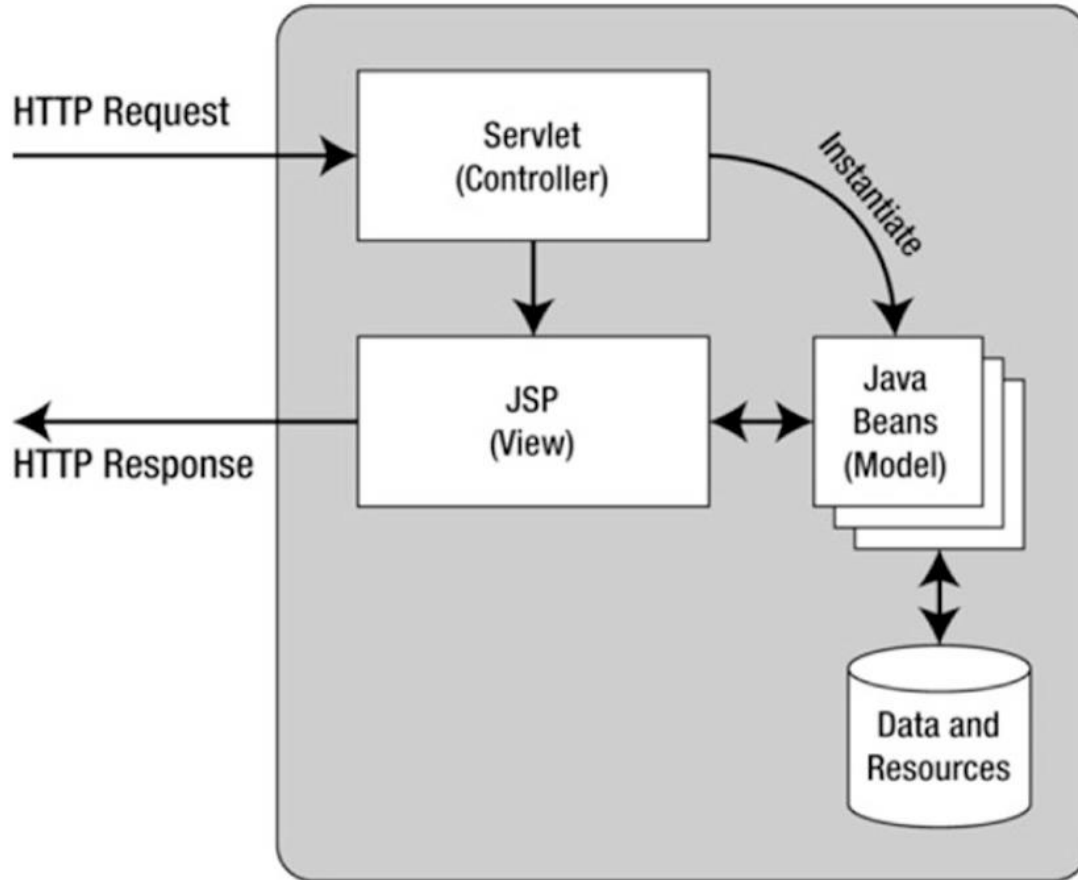
```
        doGet(request, response);
```

```
    }
```

Model 1 Architecture



Model 2 Architecture aka MVC



Web Services

Web Services

- **Web Services (WS)** – evolved from Remote Procedure Call (**RPC**) mechanism
 - They consist of operations, each of which should be implemented (according to best practices) as a *stateless* function call
 - Each **service operation** is a **function** call that:
 - Takes zero or more arguments (**input**)
 - Returns zero or more values (**output**)
 - The **client** is often a user application (i.e., user program) rather than a Web Browser
- **Service Oriented Architecture (SOA)**
 - An application results from integrating network-accessible **services**
 - Services may be characterized as *unassociated* and *loosely coupled*.

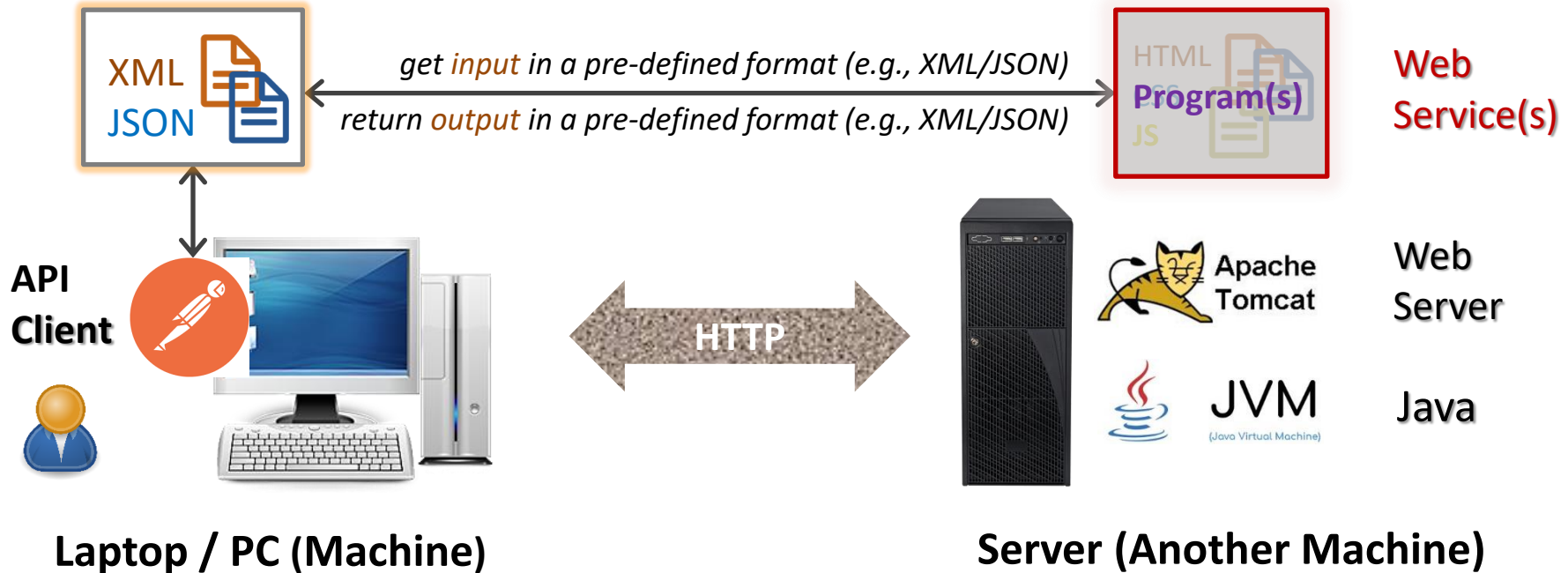
SOAP vs REST

- Web Services come *in two popular flavors*:
 - **SOAP-based Web Services**
 - SOAP is an acronym for **Simple Object Access Protocol**
 - It uses the Extensible Markup Language (**XML**) for its message format
 - It (often) relies on Hypertext Transfer Protocol (**HTTP**) for its message exchange
 - **REST-style Web Services**
 - REST stands for **Representational State Transfer** – a software architectural style !
 - It (often) uses the JavaScript Object Notation (**JSON**) as a data-interchange format
 - It (heavily) relies on Hypertext Transfer Protocol (**HTTP**)

Client – Server Model

Client-side Code (aka Front-End)

Server-side Code (aka Back-End)



XML and XML Schema

- **XML** (Extensible Markup Language) document
 - XML is a markup language and file format for storing, transmitting, and reconstructing data.

```
1  <?xml version="1.0"?>
2  <birds>
3      <owl id="1201">
4          <species>Bubo bubo</species>
5          <name>Eagle Owl</name>
6          <region>Eurasia</region>
7      </owl>
8      <owl id="1202">
9          <species>Strix occidentalis</species>
10         <name>Spotted Owl</name>
11         <region>North America</region>
12     </owl>
13 </birds>
```

- XML uses **tags** to define XML Elements
- XML elements can contain:
 - Text
 - Attributes
 - Other XML Elements

<https://www.etl-tools.com/articles/transforming-xml-data.html>

- XML Schema Definition (**XSD**) document
 - An XML document that describes the structure of an XML document

Well-formed vs Valid XML docs

- An XML document is **well-formed** when it follows certain syntactical rules outlined in the XML specification:
 - XML documents can have **only one root** element.
 - Every **start-tag** must have a matching **end-tag**, or be a self-closing tag.
 - Elements must be **properly nested**. XML tags can't overlap.
 - Element and Attribute names must obey **XML naming conventions**.
 - **Text** should properly deal with special characters.
- An XML document is **valid**, when it is **well-formed** and it **complies with** a DTD or XML schema (XSD) that define the structure of the XML document.

JSON and JSON Schema

- **JSON** (JavaScript Object Notation) document

- It is an **open data interchange format** that is readable by both humans and machines.

```
1 {  
2   "string": "Hi",  
3   "number": 2.5,  
4   "boolean": true,  
5   "null": null,  
6   "object": { "name": "Kyle", "age": 24 },  
7   "array": ["Hello", 5, false, null, { "key": "value", "number": 6 }],  
8   "arrayOfObjects": [  
9     { "name": "Jerry", "age": 28 },  
10    { "name": "Sally", "age": 26 }  
11  ]  
12 }  
13
```

- JSON is built on two data structures:
 - **Object**: A collection of name/value pairs.
 - **Array**: An ordered list of values.

- JSON Primitive Data Types:

- **Strings** (including Characters, surrounded by double quotes)
- **Numbers** (including Integers and Real Numbers)
- **Booleans** (i.e., true or false) and **null**

- **JSON Schema** document

- It is a JSON document that describes the structure of a JSON document

JSON Objects vs JavaScript Objects

- The **JSON** syntax is a *subset of* the **JavaScript** syntax (for Objects & Arrays) !
- **Key:**
 - In a **JSON Object** a key must be a **string** written with **double quotes** ("key")
 - In a **JavaScript Object**, a key can be a **string** with either **single** or **double quotes**, **number**, or **identifier name**
- **Value:**
 - In a **JSON Object** a value can be a **string**, a **number**, a **boolean**, an **object**, an **array** or **null**
 - In a **JavaScript Object**, a value can also be a **function**, a **date** or **undefined**

JSON Object vs JavaScript Object: <https://medium.com/geekculture/json-object-v-javascript-object-f00ae788cc1f>

```
---  
# A sample yaml file  
company: spacelift  
domain:  
  - devops  
  - devsecops  
tutorial:  
  - yaml:  
    name: "YAML Ain't Markup Language"  
    type: awesome  
    born: 2001  
  - json:  
    name: JavaScript Object Notation  
    type: great  
    born: 2001  
  - xml:  
    name: Extensible Markup Language  
    type: good  
    born: 1996  
author: omkarbirade  
published: true
```

YAML

- A YAML file starts with three dashes (---).
- Mappings vs Sequences (Lists)
Mappings (:) are used to associate key/value pairs
Sequences are represented by using a hyphen (-) & space.
The order matters with sequences but **not** with mappings.
- String Literals
We should quote them when they contain a value that can be mistaken for a special character.
- Comments
The YAML file supports comments, which start with hash character (#)

XML vs JSON vs YAML

- Data representation using 3 different technologies

XML	JSON	YAML
<pre><Servers> <Server> <name>Server1</name> <owner>John</owner> <created>123456</created> <status>active</status> </Server> </Servers></pre>	<pre>{ Servers: [{ name: Server1, owner: John, created: 123456, status: active }] }</pre>	<pre>Servers: - name: Server1 owner: John created: 123456 status: active</pre>

- JSON (and even XML) documents can be easily **parsed**
- Humans can easily **read** a YAML file
- XML and YAML supports **comments**

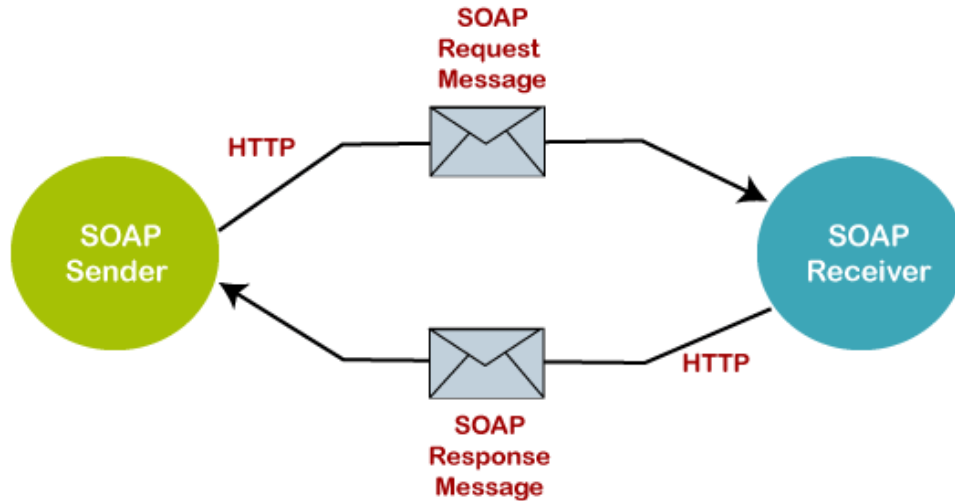
XML	JSON	YAML
eXtensible Markup Language	JavaScript Object Notation	YAML Ain't Markup Language
Markup Language	Data Interchange	Data Interchange
1996	2002	2006
A little complex	Easy to read	Easier to read
Slow	Fast	Fast
Tree Structure	Map Structure	Map Structure
.xml	.json	.yaml

Comparison between XML Vs JSON Vs YAML: <https://www.devopsschool.com/blog/comparison-between-xml-vs-json-vs-yaml/>

SOAP Web Services

SOAP Web Services

- Simple Object Access Protocol (**SOAP**) is based on the Expressive Markup Language (**XML**)



Both SOAP Request & Response are **XML** messages (aka **Envelops**)

<https://jouneidraza.medium.com/understanding-web-services-a-beginners-guide-for-soap-rest-and-webhooks-ffcf48de1ddd>

SOAP envelope in a POST HTTP message

```
POST /travelservice
```

```
SOAPAction: "http://www.acme-travel.com/checkin"
```

```
Content-Type: text/xml; charset="utf-8"
```

```
Content-Length: nnnn
```

```
• <SOAP:Envelope xmlns:SOAP= "http://schemas.xmlsoap.org/soap/envelope/">  
  • <SOAP:Body>  
    • <et:eTicket xmlns:et="http://www.acme-travel.com/eticket/schema">  
      <et:passengerName first="Joe" last="Smith"/>  
      <et:flightInfo airlineName="AA" flightNumber="1111"  
        departureDate="2002-01-01" departureTime="1905"/>  
    • </et:eTicket>  
  • </SOAP:Body>  
• </SOAP:Envelope>
```

HTTP REQUEST

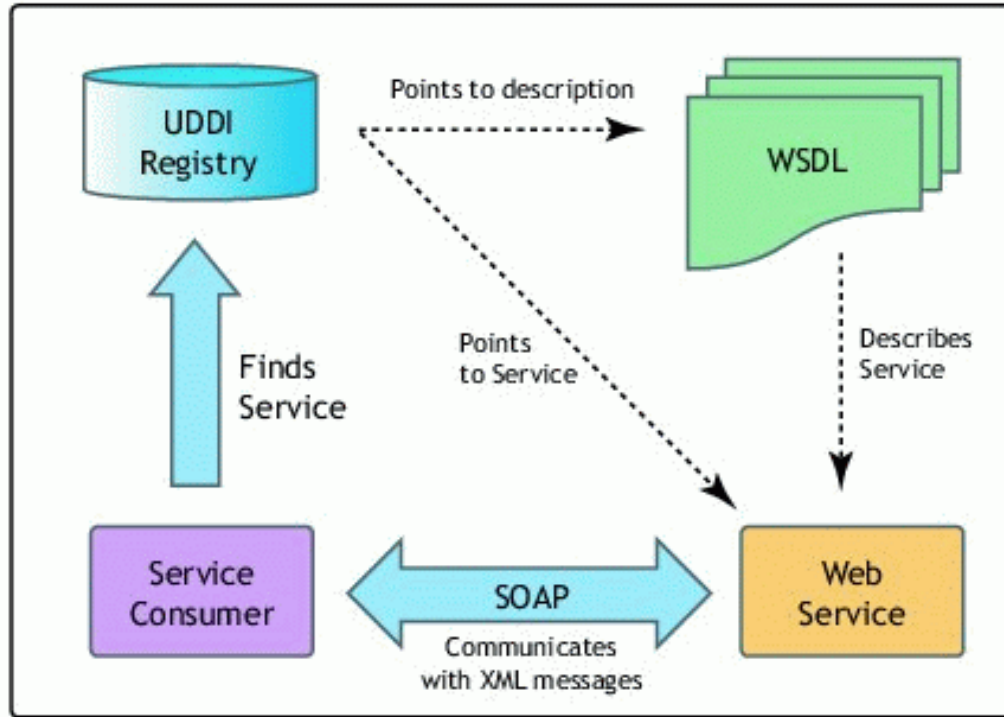
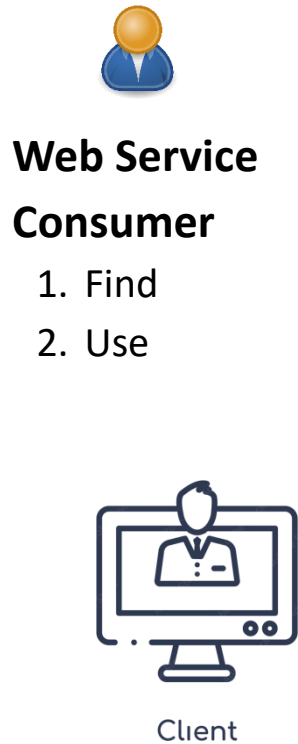
method	sp	URL	sp	Version	cr	If
header field name:	sp	value	cr	If		
header field name:	sp	value	cr	If		
cr	If					

XML

SOAP Web Services Framework

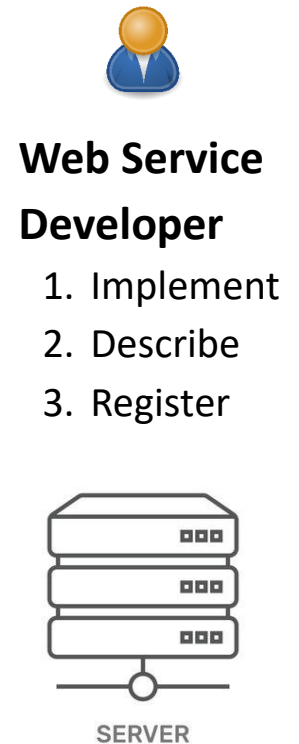
- The Web Services framework is divided into three areas:
 - Communication Protocols → SOAP
 - An XML-based **protocol** that describes message structure and process
 - Service Description → WSDL
 - An XML document that **describes** the Web Service
 - Service Discovery → UDDI
 - Defines a way to **publish and discover** information about Web services

Building Web Services-Enabled Applications



<https://support.novell.com/techcenter/articles/dnd20030304.html>

Internet Applications



Service Provider

SOAP

- **Simple Object Access Protocol (SOAP)**

- SOAP is an XML-based protocol for messaging and remote procedures calls (RPCs)
- SOAP *works on existing application layer protocols such as HTTP and SMTP.*

- **SOAP Message (aka Envelope)**

```
<SOAP:Envelope xmlns:SOAP="http://schemas.xmlsoap.org/soap/envelope/">
  <SOAP:Header>
    <!-- content of header goes here -->
  </SOAP:Header>
  <SOAP:Body>
    <!-- content of body goes here -->
  </SOAP:Body>
</SOAP:Envelope>
```

- A SOAP message consists of a Header and a Body
- The **header** defines the actors (i.e., who should process the message)
- The **body** includes the name of the remote service and the given input data

WSDL

- **Web Service Description Language (WSDL)**
 - It is used for the development of an **XML** document (it is called WSDL document) that describes the **interface** for the **web service** to the outside world.
- **WSDL document**
 - A WSDL document gives or defines the details of any web services.
 - The **data types** exchanged between the client and the web service (with XML schema).
 - What **operations** the web service has, and what messages are exchanged for each one.
 - How the web service is accessed over the network (e.g, HTTP **protocol**).
 - Where the web service can be accessed on the network (i.e., **URL**).

WSDL 2.0 - Overview: <https://jenkov.com/tutorials/wsdl/overview.html>

UDDI

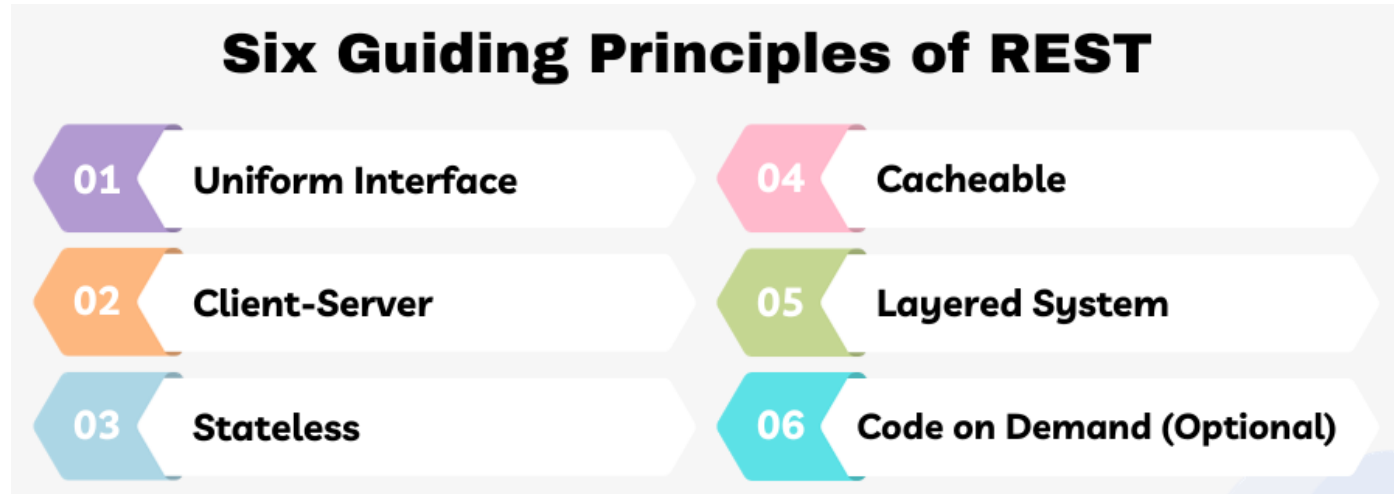
- **Universal Description Discovery and Integration (UDDI)**
 - UDDI is a specification for a distributed registry of web services.
 - It is an XML-based standard to describe, publish and find information about web services.
 - UDDI **Data Model** is an XML Schema for describing businesses and web services.

It offers users a unified and systematic way to **find** service providers through a centralized registry of services that is roughly equivalent to an automated online “phone directory” of Web services.

REST Web Services

REST (Representation State Transfer)

- REST is *a software architectural style* that *defines a set of constraints* to be used for creating Web services



The Six Guiding Principles of REST: <https://restfulapi.net/> , <https://blog.dreamfactory.com/rest-apis-an-overview-of-basic-principles>

RESTfull Web Services

- **REST Web Services** – REST stands for Representation State Transfer
 - **Web Resources** are accessible using HTTP
 - They can be identified using their **URI** $\neq$ URL
 - e.g., <http://.../webapp/users> , <http://.../webapp/users/1> or <http://.../webapp/users?id=1>
 - They have at least one representation (e.g., text/html **MIME** type)
 - Web Resources have **state**
 - A useful representation must capture resource's stated
- **Web Service Client:**
 - Specify Web Resource by providing their **URI** (as part of a URL)
 - Indicate using HTTP **verbs** what the client want to do (e.g., create a new resource)

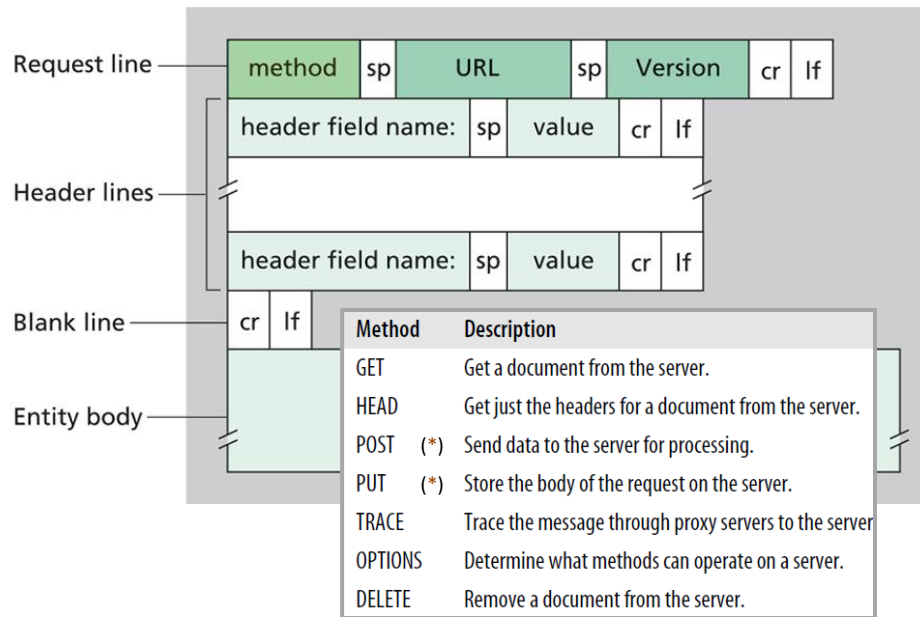
Uniform Resource Locator (URL)

`http://<host>:<port>/<path>?query#frag`

- **host**: machine **name** or **IP** address
- **port**: **number** (optional) – 80 on absence
- **path**: the resource **location** on server
- **query**: "**name=value**" pairs (separated by "&" characters)
- **frag**: points to a part or fragment of a document (optional)
(i.e., element with the given ID)

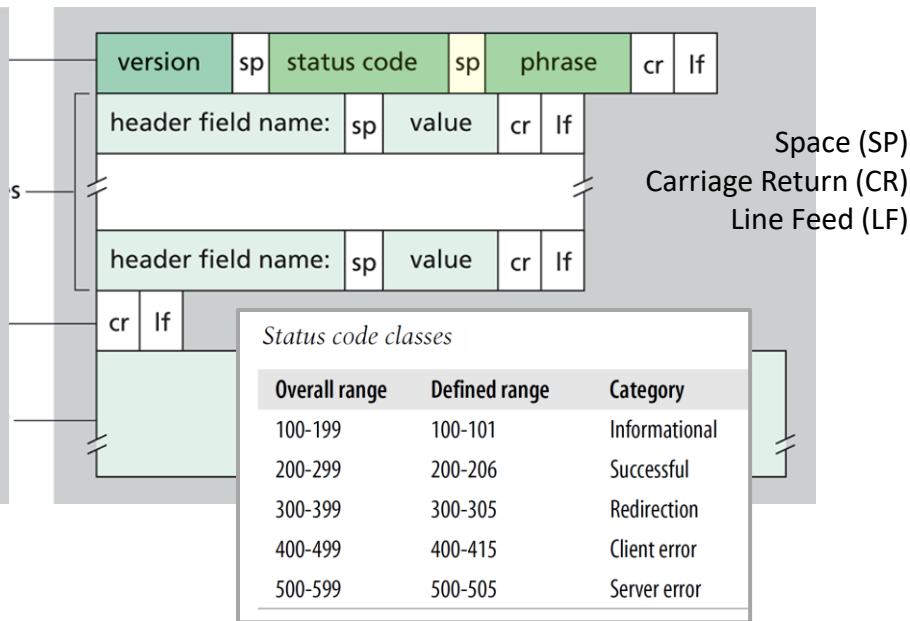
HTTP Messages

- HTTP Request Message



(*) Message body is necessary

- HTTP Response Message



HTTP as an API

- HTTP acts as an **API** and not simply as a transport protocol.
- HTTP request **methods (aka verbs)** and **MIME types**

HTTP verb	CRUD operation	DATA type	MIME type
POST	Created	HTML	text/html
GET	Read	XML	text/xml or
PUT	Update	XML	application/xml
DELETE	Delete	JSON	application/json

HTTP request:

```
GET /webapp/serv/users HTTP/1.1
Host: localhost:8080
Accept: application/json
```

HTTP response:

```
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: ...

{ ... }
```

- HTTP response **status codes**
 - **2xx** for **success**, **4xx** and **5xx** for **failure**

A key guiding principle of the **RESTful style** is to **respect the original meaning of the HTTP verbs !**

Java Servlet

REST-style Web Service using Java

- **Using Servlet API (i.e., HttpServlet)**
 - An HttpServlet has different Java **methods** for each HTTP verb (method)
 - doGet(...), doPost(...), doPut(...), doDelete(...), etc.
 - It provide access to request and response through the respective Java Objects, including **request payload** (body), **response content-type** and **status**.
- **Using Jersey (or any other Framework, such as Spring Boot)**
 - It provides an implementation of **JAX-RS** API annotations

REST-style Service using HttpServlet

```
@WebServlet("/SiteUsers")
public class SiteUsers extends HttpServlet {
    /* Retrieve Data */
    protected void doGet(HttpServletRequest request, HttpServletResponse response) {
        // Response
        final List<UserData> dataList = new ArrayList<UserData>();
        dataList.add(new UserData(1, "str1", Boolean.TRUE));
        dataList.add(new UserData(2, "str2", Boolean.FALSE));
        // Send Data in the HTTP Response Body in JSON format
        ServletUtil.sendResponseData(dataList, response);
    }

    /* Insert Data */
    public void doPut(HttpServletRequest request, HttpServletResponse response) {
        // Get User Data from the JSON existing in the HTTP Request Body
        UserData newUserData = ServletUtil.getRequestData(UserData.class, request);
        // Send HTTP response with the appropriate status and/or JSON data
        if (newUserData.id == -1) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST, "Negative Id !");
        } else {
            UserData output = new UserData(newUserData.id, null, null);
            ServletUtil.sendResponseData(output, response);
        }
    }
}
```

Web Resource: **/SiteUsers**

Service #1
Provides ALL
Site Users

Service #2
Inserts a
NEW User

Helper Methods

```
public class ServletUtil {  
    public static <T> T getRequestData(Class<T> cls, HttpServletRequest request) throws IOException {  
        // Get HTTP Request Body as (JSON) String  
        final StringBuilder sb = new StringBuilder();  
        try(BufferedReader reader = request.getReader()){  
            String line;  
            while ((line = reader.readLine()) != null){  
                sb.append(line);  
            }  
        }  
        final String payload = sb.toString();  
        // Convert (JSON) String to Object  
        final T t = jsonStrToObj(cls, payload);  
        return t;  
    }  
  
    public static void sendResponseData(Object obj, HttpServletResponse response) throws IOException {  
        // Set HTTP Response Content-Type  
        response.setContentType("text/json");  
        // Write JSON (String) in the HTTP Response Body  
        PrintWriter out = response.getWriter();  
        out.println(objToJsonStr(obj));  
        out.flush();  
        out.close();  
    }  
  
    private static <T> T jsonStrToObj(Class<T> cls, String str) { return new Gson().fromJson(str, cls); }  
    private static String objToJsonStr(Object obj) { return new Gson().toJson(obj); }  
}
```

Get HTTP request body (i.e., **JSON**) in a separate String and convert it to the corresponding Java **Object**

Convert Java **Object** to **JSON** and place it in the HTTP response body

Gson
Library

Python FastAPI

FastAPI

- **FastAPI** is a modern, fast (high-performance), **web framework** for building APIs with Python
 - FastAPI depends on **Pydantic** and **Starlette**.
 - Pydantic is the most widely used **data validation library** for Python.
 - Starlette is a **lightweight ASGI framework/toolkit**.
 - Built around **async/await** for high concurrency, suitable for real-time and scalable applications.
 - Generates interactive **documentation** (Swagger UI, ReDoc) automatically for easy testing.

FastAPI App and Endpoints

- Create a FastAPI **App**

```
from fastapi import FastAPI  
app = FastAPI()
```

- Create one or more **Endpoints**

- An endpoint is a **URL** that clients can send requests to, and the server will respond with the appropriate data.

```
@app.get("/hello")  
def read_root():  
    return {"Hello": "World"}
```

URL: `http://127.0.0.1:8000/hello`

Python: `root()` → **JSON:** ...

Endpoint – Input Data

- We can send input data through:

- HTTP Request – **URL – Path**

- Example: GET .../services/**users/1**

- HTTP Request – **URL – Query String**

- Example: GET .../services/**users?start=0&limit=10**

- HTTP Request – **Body**

- Example: POST .../services/**users**

Header: content-type: **application/json**

Body: **{"first-name": "...", "last-name": "...", ... }**

Endpoints and Path parameters

- To create an **endpoint** in FastAPI, we can use the `@app.get`, `@app.post`, `@app.put`, and `@app.delete` decorators.
- You can declare **path "parameters"** with the same syntax used by Python *format strings* (i.e., using curly brackets)
- You can declare the **type** of a "path parameter" in the function, using standard Python *type annotations*.

```
@app.get("/items/{item_id}")  
async def read_item(item_id: int):  
    return {"item_id": item_id}
```

URL: `http://127.0.0.1:8000/items/3`

Python: `read_item(3)`

Query Parameters

- When you declare other function parameters that are not part of the path parameters, they are automatically interpreted as **"query" parameters**.

```
fake_items_db = [{"item_name": "Foo"}, {"item_name": "Bar"}, {"item_name": "Baz"}]
```

```
@app.get("/items/")
async def read_item(skip: int = 0, limit: int = 10):
    return fake_items_db[skip : skip + limit]
```

URL: `http://127.0.0.1:8000/items/?skip=5&limit=20`

Python: `read_item(5, 20)`

FastAPI allows you to declare additional information and validation for your parameters.

Request Body and Data Validation

- To send data in the HTTP body you should often "make" a **POST** request
- To declare a request body, you use **Pydantic** models

```
from pydantic import BaseModel

class Item(BaseModel):
    name: str
    description: str | None = None
    price: float
    tax: float | None = None

@app.post("/items/")
async def create_item(item: Item):
    return item
```

FastAPI:

- Reads the body of the request as **JSON**.
- Makes necessary data type conversions (if needed).
- **Validates** the data.
- Gives the received data in the parameter item of your method.
- **Generates JSON Schema** definitions for your model. It will be part of the generated **OpenAPI** schema.

Response Model

- You can declare the response type by annotating the **return type** of the path operation function. However, it is generally better to define it using the `response_model` parameter.

```
class UserIn(BaseModel):
    username: str
    password: str
    email: EmailStr
    full_name: str | None = None

class UserOut(BaseModel):
    username: str
    email: EmailStr
    full_name: str | None = None

@app.post("/user/", response_model=UserOut)
async def create_user(user: UserIn) -> Any:
    return user
```

FastAPI:

- **Limit** and **Filter** the output data to what is defined in the return type.
- **Validate** the returned data.
- **Serialize** the returned data to JSON using Pydantic.
- Add a **JSON Schema** for the response, in the **OpenAPI** path operation.

Uvicorn

- **ASGI** stands for **A**synchronous **S**erver **G**ateway **I**nterface
 - It is a standard interface between **network protocol services** and **Python applications**.
 - It can handle multiple common protocol types, including HTTP, HTTP2, and WebSocket.
 - ASGI protocol, available at <https://asgi.readthedocs.io/>
- **Uvicorn is an ASGI web server implementation for Python.**
 - It is a high-performance ASGI server implemented with uvloop and httptools, which can handle HTTP requests asynchronously.

Start Uvicorn Server

- **Running programmatically – Example**

main.py

```
import uvicorn

async def app(scope, receive, send):
    ...

if __name__ == "__main__":
    uvicorn.run("main:app", port=5000, log_level="info")
```

- **Command Line:**

```
uvicorn main:app --port 5000 --log-level info
```

main: The Python file/module name (main.py)

app: The FastAPI application object inside that file

host. Use --host **0.0.0.0** to make the application available on your local network.

Homework

- Create a **REST service** that enable users to manage the Objects residing in an in-memory Python **List**
 - We need 4 methods, one for each **CRUD** operation
- Tested using automatically generated **Swagger UI**

Nginx

- An **HTTP web server**, reverse proxy, content cache, load balancer, TCP/UDP proxy server, and mail proxy server.
 - We can use it for serving **static and index files**
 - We can use it as a proxy in front of your **Uvicorn processes**
- In Nginx, we have 3 **common elements**:
 - **Directives**: instructions that are used to configure the web server
 - **Includes**: additional files that NGINX should use for configuration
 - **Context**: specific section of NGINX configuration you are working with

Useful Links

- **FastAPI** Site: <https://fastapi.tiangolo.com/>
 - Tutorial: <https://fastapi.tiangolo.com/tutorial>
- **Uvicorn** Site: <https://uvicorn.dev/>
 - Settings: <https://uvicorn.dev/settings/>
- **Nginx** Site: <https://nginx.org/>
 - Tutorial: https://nginx.org/en/docs/beginners_guide.html

SQLite

SQLite

- A database engine written in the **C programming language**
- A **library** that software developers **embed** in their apps
- It generally follows PostgreSQL syntax but **does not enforce type checking** by default (dynamic type system).



- **SQLite DB:**
 - SQLite Home Page: <https://www.sqlite.org/>
- **SQLite UI** (e.g., Browser):
 - SQLiteStudio: <https://sqlitestudio.pl/>
 - SQLite Browser: <https://sqlitebrowser.org/dl/>

DB Schema & SQL Queries

- **Database Schema – DDL (Data Definition Language)**

- Example:

```
CREATE TABLE user (  
    u_id INTEGER PRIMARY KEY,  
    u_name TEXT NOT NULL UNIQUE,  
    u_age INTEGER NOT NULL  
);
```

- **Database «Queries» – SQL (Structured Query Language)**

- Example 1:

```
INSERT INTO user VALUES (1, 'User Name', 32)
```
 - Example 2:

```
SELECT * FROM user
```

SQLite Classes & Datatypes

- Each value stored in SQLite database has one of the following **storage classes**:
 - NULL
 - INTEGER
 - REAL
 - TEXT
 - BLOB
- SQLite does not have a separate **Boolean** storage class.
 - Boolean values are stored as integers, 0 (false) and 1 (true).
- SQLite does not have a storage class set aside for storing **dates and/or times**.
 - We can store dates & times as TEXT, REAL, or INTEGER values
- A storage class is more general than a **datatype**.
 - For example, the INTEGER storage class includes 7 different integer datatypes of different lengths.

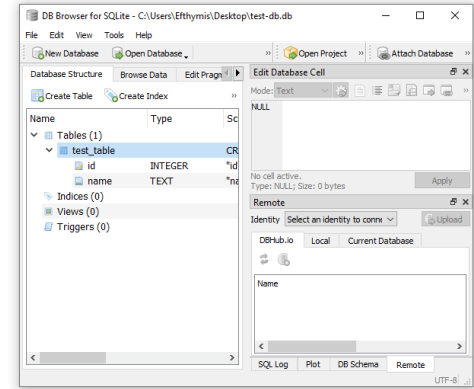
SQLite Datatypes: <https://sqlite.org/datatype3.html>

Any column in an SQLite v3 DB (except column for primary key), may be used to store *a value of any storage class* !

Homework

- SQLite Database
 - Create a simple **SQLite DB** in your PC/Desktop with a simple table using a software program
- SQLite Queries
 - Run two different type of **SQL queries**
 - An **insert** SQL query & A **select** SQL query
- **What if ?**
 - We try to store an integer value in a text field ?

DB Browser for SQLite



Open/Close Database Connection

```
import sqlite3
```

- Create a new database or **connect** to an existing one

```
connection = sqlite3.connect('my_database.db')
```

to force an SQLite database
to exist purely in memory:
':memory:'

- It's important to **close** the connection to free up any resources.

```
connection.close()
```

- You can use the **with statement** to handle database connections

Executed Command – Create a Table

- The **cursor** is responsible for executing SQL commands and queries on the database

```
cursor = connection.cursor()
```

- We can **execute** the CREATE TABLE SQL command in Python using the sqlite3 library.

```
create_table_query = 'CREATE TABLE IF NOT EXISTS ... ( ... );'  
  
cursor.execute(create_table_query)  
  
connection.commit()
```

Insert Data into a Table

- Insert a **Single Records**

Use parameterized queries to avoid **SQL injection**

```
insert_query = ' INSERT INTO ... ( ... ) VALUES (?, ... , ?); '
```

```
data = ( ... , ... , ... )
```

```
cursor.execute(insert_query, data)
```

```
connection.commit()
```

The INSERT statement implicitly opens a **transaction**, which needs to be committed before changes are saved in the database

- Insert **Multiple Records**

- You can use the **executemany()** method. It takes (a) the insert query and (b) a list of tuples.

Query Data

- Retrieve **all records**

```
select_query = "SELECT ... FROM ... "  
  
cursor.execute(select_query)  
  
all_ = cursor.fetchall()
```

Each row is a **tuple**, matching the columns selected in the query.

- Retrieve them **one by one**

```
row = cursor.fetchone()
```

If no row is available will return **None**

Update / Delete Data

- **Delete** a few Records

```
delete_query = ' DELETE FROM ... WHERE ... '  
  
cursor.execute(delete_query, ( ... ) )  
  
connection.commit()
```

– We can find the rows affected using cursor as follows: `cursor.rowcount`

- **Update** a few Records following a similar process

Manage Transactions

- **Starting** a Transaction

```
cursor.execute("BEGIN;")
```

- **Rolling** Back a Transaction

```
connection.rollback()
```

- **Commit** a Transaction

```
connection.commit()
```

Example 1 – Insert a Row in a Table

```
# Use 'with' to open and close the connection automatically
with sqlite3.connect('my_database.db') as connection:

    cursor = connection.cursor()

    insert_query = '''
    INSERT INTO Students (name, age, email)
    VALUES (?, ?, ?);
    '''

    student_data = ('Jane Doe', 23, 'jane@example.com')

    cursor.execute(insert_query, student_data)

    connection.commit()

    print("Record inserted successfully!")
```

Example 2 – Get Rows from a Table

```
with sqlite3.connect('my_database.db') as connection:

    cursor = connection.cursor()

    # Write the SQL command to select all records from the Students table
    select_query = "SELECT * FROM Students;"

    # Execute the SQL command
    cursor.execute(select_query)

    # Fetch all records
    all_students = cursor.fetchall()

    # Display results in the terminal
    print("All Students:")
    for student in all_students:
        print(student)
```

Homework

- Create a Python script that:
 - **Create** an in-memory **SQLite database**
 - **Create** a **table** with one or more fields
 - **Insert** two or more **records**
 - **View** the **data** existing in table
 - **Remove/Update** a record and view table data

Placeholders

- An SQL statement may use one of two kinds of placeholders:
 - Question marks (?)
 - Named Placeholders (:name) – Example:

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")
data = (
    {"name": "C", "year": 1972},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)
```

For the named style, parameters must be an instance of a [dict](#).

Adapter and Converter

Python vs SQLite data types

- **Adapter:** a function to adapt custom Python types to SQLite values.
- **Converter:** a function to convert SQLite values to custom Python types.
- Both adapter and converter should be **registered**.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

def adapt_point(point):
    return f"{point.x};{point.y}"

def convert_point(s):
    x, y = list(map(float, s.split(b";")))
    return Point(x, y)

# Register the adapter and converter
sqlite3.register_adapter(Point, adapt_point)
sqlite3.register_converter("point", convert_point)
```


Example

```
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_COLNAMES)
cur = con.execute("CREATE TABLE test(p)")
cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute('SELECT p AS "p [point]" FROM test')
print("with column names:", cur.fetchone()[0])

cur.close()
con.close()
```

Create Database using DB schema

- Use `executescript()` to execute multiple SQL statements.

```
connection = sqlite3.connect("database.db")

cursor = connection.cursor()

with open("database_schema.sql", "r") as f:
    sql_script = f.read()

cursor.executescript(sql_script)

connection.commit()

connection.close()
```

Useful Links

- sqlite3 — DB-API 2.0 interface for SQLite databases

<https://docs.python.org/3/library/sqlite3.html>

- How to Work with SQLite in Python

<https://www.freecodecamp.org/news/work-with-sqlite-in-python-handbook/>

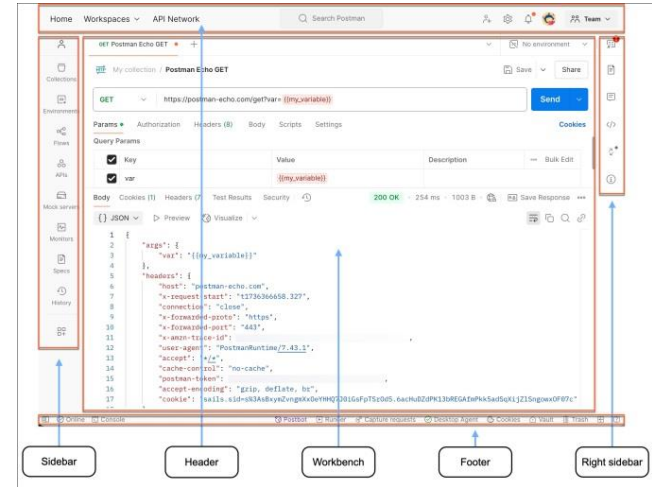
- Introduction to SQLite in Python

<https://www.geeksforgeeks.org/python/introduction-to-sqlite-in-python/>

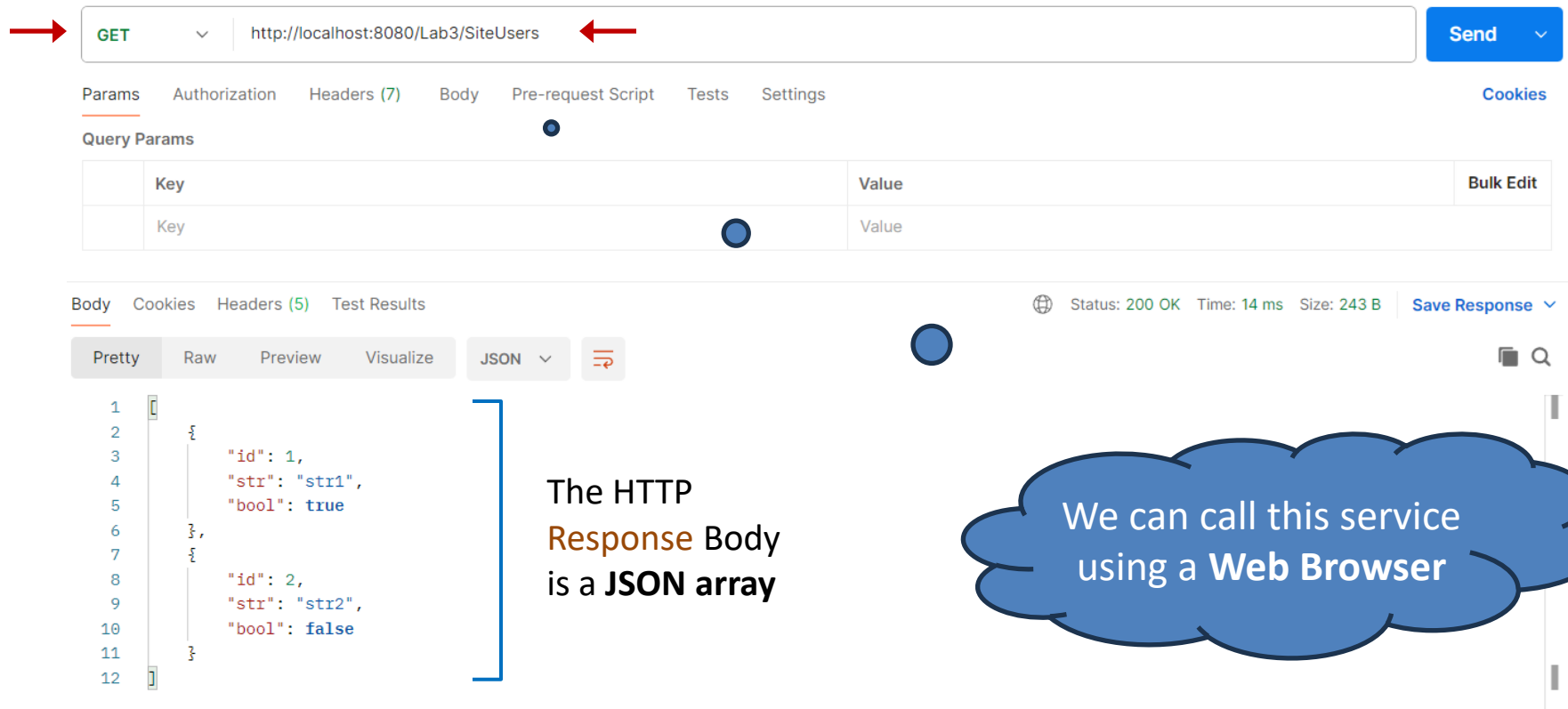
Client-side

(REST) Web Service Client

- Linux **curl** command
 - `curl [options] [URL]`
- **Postman:** <https://www.postman.com/>
 - Enables you to create and send API requests.
- **Web Application Code**
 - Client-side – **JavaScript** : using **fetch** API
 - Server-side – **Java, Python , etc.**
- **Web Browser (e.g., GET requests) & REST Service Web Page (if exists)**



Postman – Retrieve All Site Users



→ GET http://localhost:8080/Lab3/SiteUsers ← Send

Params Authorization Headers (7) Body Pre-request Script Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

Body Cookies Headers (5) Test Results Status: 200 OK Time: 14 ms Size: 243 B Save Response

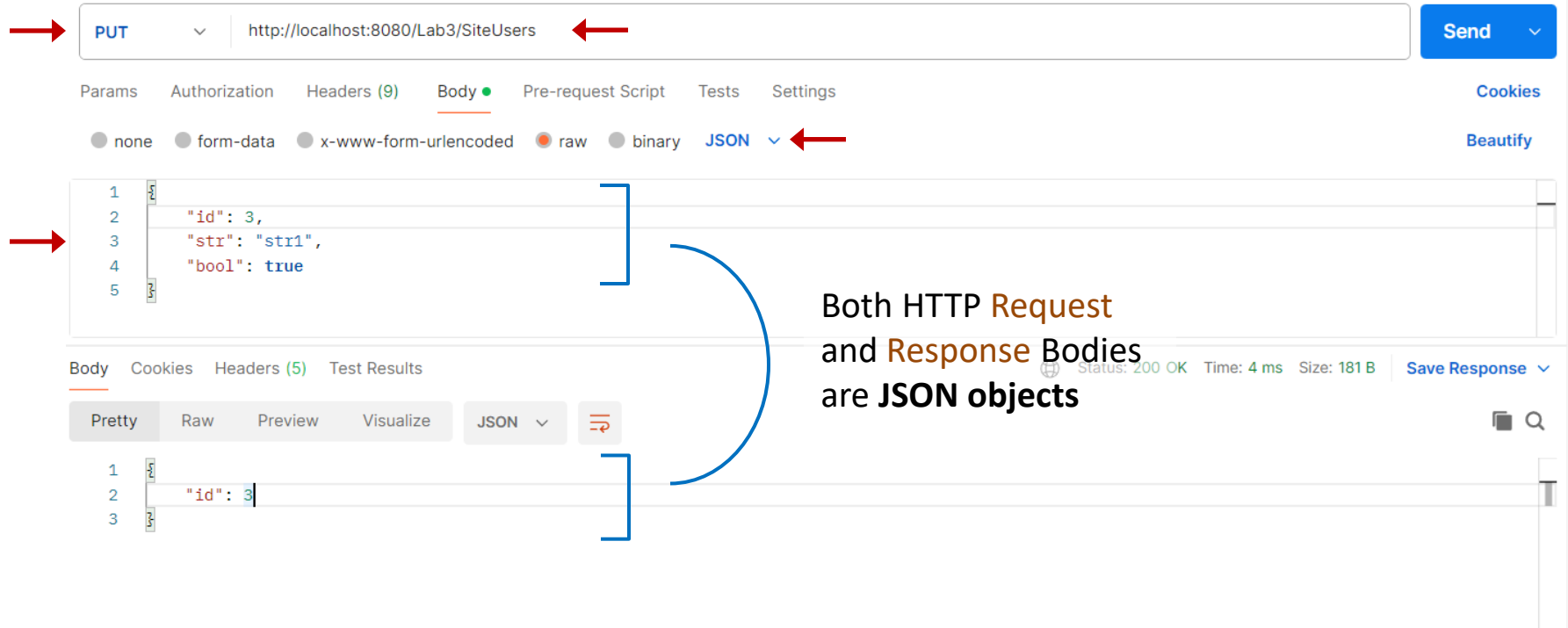
Pretty Raw Preview Visualize JSON

```
1 [
2   {
3     "id": 1,
4     "str": "str1",
5     "bool": true
6   },
7   {
8     "id": 2,
9     "str": "str2",
10    "bool": false
11  }
12 ]
```

The HTTP Response Body is a JSON array

We can call this service using a Web Browser

Postman – Insert a New Site User



PUT http://localhost:8080/Lab3/SiteUsers

Params Authorization Headers (9) **Body** Pre-request Script Tests Settings

none form-data x-www-form-urlencoded **raw** binary **JSON**

```
1 {
2   "id": 3,
3   "str": "str1",
4   "bool": true
5 }
```

Body Cookies Headers (5) Test Results

Pretty Raw Preview Visualize **JSON**

```
1 {
2   "id": 3
3 }
```

Status: 200 OK Time: 4 ms Size: 181 B Save Response

Both HTTP Request and Response Bodies are JSON objects

Web Browser and JavaScript Code

- The web browser **retrieves** relevant files from a web server (using HTTP) and then **displays** the web page on the user's screen.

<script> : is used to define client-side JavaScript (JS) code
(i.e., either the JS code or points to the JS file with JS code)

- When the HTML parser encounters a **<script>** element, **it must, by default, run the script**, just to be sure that it doesn't output any HTML before it can resume parsing and rendering the document.

Interaction with the User

- Events and Event Handlers

- JavaScript programs register **callback functions** (aka **event handlers**) for specified types of events in specified contexts, and the web browser invokes those functions whenever the specified **events** occur.

```
const s = document.querySelector("#form input[type=submit]");
s.addEventListener("click", (e)=>{
    const p1 = document.querySelector("#form2 input[name=new-pass1]");
    const p2 = document.querySelector("#form2 input[name=new-pass2]");
    if ( p1.value.trim() && p1.value === p2.value) {
        // Submit Form...
    } else {
        alert("Passwords given do not match !");
        e.preventDefault();
    }
});
```

Find HTML Element with ID "form" and then search for HTML elements of type input with attribute "type" and value "submit"

`console.log(Boolean(""));`
`false`

Networking – The fetch() method

- The **fetch()** method defines a Promise-based API for making HTTP and HTTPS requests
 - The **fetch()** method returns a **promise** !

```
fetch("/api/users/current")           // Make an HTTP (or HTTPS) GET request
  .then(response => response.json())  // Parse its body as a JSON object
  .then(currentUser => {              // Then process that parsed object
    displayUserInfo(currentUser);
  });
```

Promises

- **A Promise** is an object that **represents the future result** of *a single asynchronous computation*.
 - They **cannot** be used to represent repeated asynchronous computations !
- The Promise object defines a method: **then()**
 - **When** the asynchronous compulsion completed, the result is given to the function that we passed to method **then()**
- **Chaining Promises**
 - The method **then()** **returns a new Promise immediately**.
 - We can create a **sequence of asynchronous operations** as a linear chain of **then()** method invocations

Dealing with Problems

- Promise-related errors are typically handled by adding a `.catch()` method invocation to a Promise chain.
 - The `.catch()` method of a Promise is simply a shorthand way to call `.then()` with null as the 1st argument and an error-handling callback as the 2nd argument.

```
...  
.then( ... )  
.catch(e => { console.error(e); });
```

- Promise objects also define a `.finally()` method whose purpose is *similar to the finally clause in a try/catch/finally statement*.
 - The return value of a `.finally()` callback is a Promise and it is generally ignored.

REST Service call from JavaScript Code

```
1 document.addEventListener("DOMContentLoaded", function(){
2
3     let remove_button = document.querySelector("#remove_button");
4     remove_button.addEventListener("click", function(e) {
5         let user_id = document.querySelector("#user_id");
6         let mydiv = document.querySelector("#mydiv");
7         // Make HTTP Request - Request to DELETE a Web Resource
8         let web_resource_url = "SiteUsers/" + user_id.value;
9         fetch(web_resource_url, {method: "DELETE",})
10            .then(response => {
11                if (response.ok && response.headers.get("Content-Type").startsWith("text/json") ) {
12                    return response.json();
13                } else {
14                    throw new Error("Unexpected problem: " + response.status);
15                }
16            })
17            .then(json_data => {
18                // Handle JSON data
19                mydiv.innerHTML = "json_data: " + JSON.stringify(json_data);
20            })
21            .catch(error => {
22                console.log("Fetching problem: " + error);
23            });
24    });
25
26 });
```

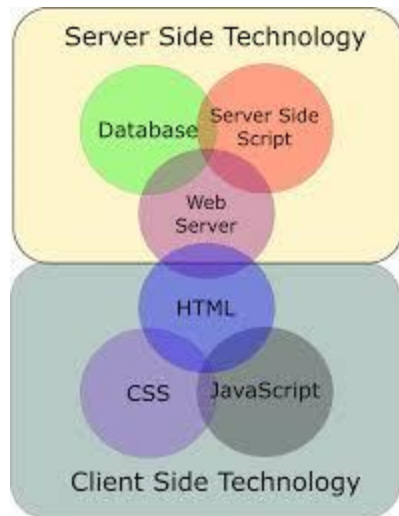
```
<!DOCTYPE html>
<html>
  <body>
    <!-- ... -->
    <input id="user_id" type="text" placeholder="uid">
    <input id="remove_button" type="button" value="Remove">
    <div id="mydiv">Nothing yet !</div>
    <script src="page.js"></script>
  </body>
</html>
```



Conclusion and Next Steps

Conclusion

- **Web Application Development**
 - Interaction between Client and Server as well as Server with RDBMS
 - Apache Tomcat, Java Servlet, JSPs, and JDBC API
- **Web Services**
 - SOAP-based Web Services
 - REST-style (Web) Services
 - Python FastAPI, Uvicorn, SQLite
- **Asynchronous JavaScript**
 - `fetch(..)`



Next Steps



Lab: 16/05/2026

announcement will be made

April 2026							May 2026						
Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday	Monday	Tuesday	Wednesday	Thursday	Friday	Saturday
			1	2	3	4						1	2
5	6	7	8	9	10	11	3	4	5	6	7	8	9
12	13	14	15	16	17	18	10	11	12	13	14	15	16
19	20	21	22	23	24	25	17	18	19	20	21	22	23
26	27	28	29	30			24	25	26	27	28	29	30
							31						

Thank You!

